Assignment 3 NLP: подробное объяснение для начинающего

Этот документ объясняет готовый ноутбук: что требовалось в задании, какие данные использованы, как устроен код, что означают результаты и что могут спросить на защите. Разбор относится к ноутбуку из коммита `fff20c33ed0601a58358165d22912e27d7422f8d`. Документ не изменяет алгоритмы и результаты ноутбука.

Как читать: сначала разделы 1–4, затем разбирайте код параллельно с ноутбуком. Номер **блока** здесь означает номер только кодовой ячейки от начала файла: всего их 21. Номер **ячейки** учитывает также Markdown-текст: всего ячеек 39. У каждого блока есть объяснение и полный исходный код с локальными номерами строк. Номера строк начинают отсчёт заново в каждом блоке; пустые строки не требуют отдельного действия. Длинные словари с текстами статей и контрольными суммами объясняются как данные, без повторного копирования статей.

Содержание

1. Что нужно сделать и как устроен ноутбук
2. Данные и библиотеки
3. Минимум Python для чтения кода
4. Как понимать обучение и метрики
5. Разбор всех кодовых блоков
6. Результаты проверенного запуска
7. Вопросы преподавателя и ответы
8. План выступления и тренировка
9. Запуск, файлы результатов и частые ошибки

1. Что нужно сделать и как устроен ноутбук

NLP — Natural Language Processing, обработка естественного языка. Компьютер получает обычный человеческий текст, а программа извлекает из него информацию.

Задание	Вход	Что требуется	Реализация
1. Тональность	Отзыв клиента	Negative / neutral / positive двумя способами	Словарь VADER; TF-IDF + Logistic Regression
2. Суммаризация	Статьи article1–article5	По 3 предложения из первых 3 файлов; общий top-6; новая сводка по 5 файлам	TF-IDF-центроид; предобученная T5-small
3. Песни	40 текстов, 4 жанра	Top-10 слов, тематические категории, редкие слова	Очистка, лемматизация и Counter

Задание	Вход	Что требуется	Реализация
4. Язык	Текст на одном из 6 языков	Сравнить признаки, очистку, модели и длину входа	8 сочетаний TF-IDF/NB/LR; fastText lid.176

В первом и четвёртом заданиях мы **обучаем классификаторы** на примерах с правильными ответами. В третьем задании обучения нет: это частотный анализ. Во втором TF-IDF ранжирует предложения, а T5 уже обучена другими исследователями: мы запускаем её, но не дообучаем.

Типичная цепочка работы:

```
Текст → очистка → числовые признаки → модель → предсказание → сравнение с правильным ответом
```

Для песен цепочка короче: `текст → слова-леммы → подсчёт → таблица и график`. Для генерации: `документы → токены → небольшие блоки → промежуточные сводки → общая сводка`.

Что представляет собой .ipynb: JSON-файл с ячейками кода, пояснениями и сохранёнными выводами. Markdown-ячейки отображают текст; кодовые ячейки выполняет Python-ядро, или kernel. Обычно запускать нужно сверху вниз: нижние ячейки используют созданные выше переменные. Сохранённый график показывает прошлый запуск; он сам по себе не означает, что текущее ядро уже обучило модель. Для собственного запуска выберите Run all.

2. Данные и библиотеки

Датасеты

Данные	Источник и назначение	Что реально используется
Twitter US Airline Sentiment	Размеченные отзывы об авиакомпаниях; Kaggle, CSV-зеркало	После очистки 13 981 отзыв, три настоящие метки
article1.txt … article5.txt	Файлы, приложенные к заданию	Встроены в ноутбук; локальные одноимённые файлы при наличии имеют приоритет
genre_lyrics_data.csv	Корпус песен: тексты AZLyrics, жанры Spotify	Country, blues, folk, hip-hop; по 10 уникальных текстов
WiLI-2018	Описание и оригинальный корпус, публичное зеркало	По 800 примеров для English, Russian, French, German, Spanish, Portuguese

Нейтральный класс не придумывается из двоичного IMDb: он уже есть в Airline Sentiment. У песен нет названий и исполнителей в выбранной CSV, поэтому идентификатором служат номер исходной строки и хеш текста. В WiLI один пример часто является абзацем, а не ровно одним предложением. Все ссылки загрузки в коде привязаны к конкретным Git-коммитам, чтобы смена основной ветки источника не меняла данные незаметно.

Что делают библиотеки

Библиотека	Простое объяснение
pathlib, os, sys	Работа с папками, настройками процесса и текущим Python
subprocess	Запуск другой программы, здесь — pip для установки пакетов
re	Поиск и замена по шаблонам: ссылки, пробелы, слова
json, hashlib	Чтение/запись структур данных; контрольные суммы файлов
collections.Counter	Счётчик вхождений слов
NumPy	Массивы чисел, сортировка индексов и операции над ними
pandas	Таблицы с колонками, фильтрацией и группировкой
scikit-learn	Разбиение данных, TF-IDF, классификаторы, метрики
NLTK	Стоп-слова, POS-теги, WordNet, разбиение на предложения, VADER
matplotlib, wordcloud	Диаграммы и облака слов
requests, urllib3	Загрузка по HTTPS, повтор запросов при временных сбоях
transformers	Токенизатор и интерфейсы экосистемы Hugging Face
PyTorch	Тензоры для входа генератора и режим без вычисления градиентов
optimum, ONNX Runtime	Выполнение экспортированной T5 на CPU
fasttext-wheel	Доступ к предобученному определителю 176 языков
rouge-score	Метрики лексического совпадения аннотаций с эталоном

Библиотека — набор готовых инструментов. Модель — обученная система с параметрами. Например, `transformers` — библиотека, а T5-small — модель. TF-IDF — метод представления текста числами; сам по себе он не является классификатором тональности.

3. Минимум Python для чтения кода

Запись	Значение и пример
`x = 5`	Присвоить значение переменной. = не проверяет равенство
`x == 5`, `x != 5`	Проверить равенство / неравенство
`'text'`, `"text"`	Строка. Оба вида кавычек допустимы
`['a', 'b']`	Список с порядком элементов
`{'word': 3}`	Словарь: ключ `word` связан со значением `3`
`{'a', 'b'}`	Множество уникальных значений; удобно проверять пересечения

Запись	Значение и пример
`def f(x): ...`	Определить функцию; её тело выполняется при вызове `f(...)`
`return result`	Вернуть результат из функции
`for x in items:`	Повторить тело цикла для каждого элемента
`if condition:`	Выполнить тело только при истинном условии
`lambda x: x.lower()`	Короткая безымянная функция для одного выражения
`[f(x) for x in xs]`	Создать список, применив f ко всем x
`a if condition else b`	Выбрать одно из двух значений
`with open(...) as f:`	Открыть файл и автоматически закрыть после блока
`try / except`	Попробовать действие и обработать указанный тип ошибки
`assert condition`	Остановить обычный запуск, если важная проверка не прошла
`f'{score:.3f}'`	Вставить число в строку с 3 знаками после точки
`**some_dict`	Распаковать словарь: добавить поля либо передать именованные аргументы
`text[:3]`, `texts[-1]`	Срез первых 3 элементов; последний элемент
`zip(a, b)`	Перебирать соответствующие элементы двух последовательностей
`enumerate(xs)`	Получать одновременно номер и элемент

Отступы — часть синтаксиса: строки внутри `def`, `if`, `for` принадлежат своему блоку. Комментарий после `#` не исполняется. `None` означает отсутствие значения; `True` и `False` — логические значения. `.method()` вызывает метод объекта; `.label` у pandas здесь обращается к колонке `label`, а не к особому Python-типу.

Почти весь табличный код можно читать как операции Excel:

```
df[df.label == 'positive']  # оставить строки с нужной меткой
df['new'] = values          # добавить колонку
df.text.map(clean)          # применить clean к каждому тексту
df.groupby('label').size() # посчитать строки каждой группы
```

В pandas `~mask` означает «не», а `mask1 & mask2` — «и» для массивов условий. Поэтому для фильтров используют скобки вокруг каждого сравнения. `.copy()` создаёт отдельную таблицу; `.reset_index(drop=True)` перенумеровывает строки.

4. Как понимать обучение и метрики

Объект — один отзыв или один текст. **Метка** — правильный ответ: например, positive. **Признак** — число, которое модель использует для решения: вес слова или символьного сочетания. **Предсказание** — ответ модели. Датасет содержит много объектов.

Train, validation, test

- **Train:** модель изучает связи между признаками и метками.
- **Validation:** выбираем настройки, например пороги VADER или C у регрессии.
- **Test:** оцениваем уже выбранный метод на неиспользованных для выбора примерах.

В задаче 1 train/validation/test примерно 60/20/20. После выбора C итоговую ML-модель переобучаем на train + validation: test по-прежнему отдельно. В задаче 4 требуется 80/20; настройки фиксированы заранее, отдельная validation не создаётся. Таблица test позволяет описать результаты разных моделей, но подбор по ней новых настроек уже сделал бы её валидационным набором.

Утечка данных — информация из проверки заранее попала в обучение. Дубликат одного отзыва в train и test может завысить результат. Обучить TF-IDF на всех текстах до разделения — тоже передать модели статистику теста. Поэтому сначала делим, затем `fit` на train; `Pipeline` помогает соблюдать порядок. Детерминированная очистка вроде lower сама по себе не учится по корпусу; её можно выполнить заранее. Но даже дедупликация и исключение конфликтной разметки меняют оцениваемую совокупность: наши метрики относятся к очищенному корпусу.

Accuracy, precision, recall, F1

Для одного класса, например positive:

- **TP:** положительный отзыв правильно назван положительным.
- **FP:** модель сказала positive, но истинная метка другая.
- **FN:** истинный positive модель отнесла в другой класс.
- **Accuracy:** число всех правильных ответов / число всех объектов.
- **Precision:** `TP / (TP + FP)` — насколько можно доверять ответу positive.
- **Recall:** `TP / (TP + FN)` — какую долю настоящих positive нашли.
- **F1:** `2 × precision × recall / (precision + recall)` — баланс двух предыдущих величин.

Пример: из 10 истинных positive нашли 6; всего сказали positive 8 раз. Тогда TP=6, FN=4, FP=2, precision=6/8=0.75, recall=6/10=0.6, F1≈0.667. Это пример для понимания формул, не данные нашего запуска.

Macro F1 — посчитать F1 каждого класса и взять обычное среднее. Каждый класс имеет одинаковый вес. Weighted F1 взвешивает их по числу объектов, а micro F1 объединяет счётчики; в обычной однозначной многоклассовой классификации micro F1 совпадает с accuracy.

Confusion matrix: строки — истинные классы, столбцы — предсказанные. Диагональ показывает правильные ответы. Вне диагонали видно, какие классы путаются. `Support` в отчёте — число истинных объектов класса, а не уверенность модели.

Bag-of-Words и TF-IDF

Bag-of-Words превращает текст в количества слов. Для словаря `[good, bad, flight]` текст `good flight flight` даёт `[1, 0, 2]`. Обычный BoW не хранит порядок слов. TF-IDF уменьшает вклад слов, встречающихся почти в каждом документе. В использованной реализации со сглаживанием:

```
idf(word) = log((1 + N) / (1 + df(word))) + 1
tf(word) = 1 + log(count(word)) для ненулевых частот при sublinear_tf=True
вес = tf × idf; затем по умолчанию L2-нормирование вектора документа
```

Здесь N — число обучающих документов; df — в скольких из них встречается слово. Большинство элементов матрицы нулевые: конкретный отзыв содержит мало слов из всего словаря. Поэтому scikit-learn использует **разреженную матрицу**, экономящую память. Словесная биграмма — два соседних слова (`not good`), символьная триграмма — три соседних символа (`ing`). Это разные признаки и разные способы учитывать локальную структуру.

5. Разбор всех кодовых блоков

Блок 1. Установка библиотек

Ячейка 2 в файле.

Зачем: подготовить Python к выполнению заданий. Установить пакет и импортировать его — разные операции: первая добавляет его в окружение, вторая даёт доступ к нему текущему коду.

Строки	Объяснение
1	Комментарий: устанавливать зависимости лучше до импорта NumPy, чтобы не смешать версии в памяти
2	Импортируем модули для настроек, сведений о Python и запуска команды
3	Читаем переменную окружения. Установка пропускается только если `NLP_SKIP_INSTALL` равна строке `"1"`. При обычном запуске её нет, значит установка выполняется
4	Запускаем `python -m pip install -q` именно текущим Python (`sys.executable`). `check_call` завершится ошибкой, если pip не справился; `-q` уменьшает журнал
5–10	Список требуемых пакетов. == закрепляет точную версию, `>=...<...` задаёт диапазон совместимых версий
11	Печатаем подсказку о перезапуске ядра при конфликте уже загруженных библиотек

Переменная `NLP_SKIP_INSTALL` использовалась при проверке в заранее подготовленном окружении. Вам не требуется её задавать. `torch>=2.2` не закрепляет верхнюю границу, поэтому в другой среде возможна иная версия PyTorch: воспроизводимость не означает абсолютную идентичность всех будущих окружений. Фактические версии проверенного запуска сохранены в конце ноутбука.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | # Установка выполняется до импорта numpy/pandas.
002 | import os, sys, subprocess
003 | if os.environ.get("NLP_SKIP_INSTALL") != "1":
```

```
004 |     subprocess.check_call([sys.executable, "-m", "pip", "install", "-q",
005 |         "numpy==1.26.4", "pandas==2.2.3", "scikit-learn==1.5.2",
006 |         "nltk==3.9.1", "matplotlib>=3.9,<4", "wordcloud>=1.9,<2",
007 |         "requests>=2.32,<3", "transformers==4.46.3", "sentencepiece>=0.2,<0.3",
008 |         "huggingface_hub>=0.26,<1", "torch>=2.2",
009 |         "optimum==1.23.3", "onnx==1.17.0", "onnxruntime==1.20.1", "fasttext-wheel==0.9.2",
010 |         "rouge-score==0.1.2"])
011 | print("Зависимости установлены. При конфликте с ранее импортированным numpy перезапустите ядро и Run
all.")
```

Блок 2. Общие инструменты, папки и вспомогательные функции

Ячейка 3 в файле.

Это техническая основа для всех заданий. Здесь ещё не обучается ни один классификатор.

Строки	Что происходит
1–4	Импортируем Path, Counter, кеширование и стандартные модули. `warnings` импортирован, но далее не используется — на результат это не влияет
5–11	Импортируем массивы, таблицы, графики, загрузки и красивый вывод Jupyter
12–18	Подключаем разбиение данных, TF-IDF, LR, NB, Pipeline, метрики и косинусное сходство
19–23	Подключаем инструменты обработки английского и облака слов
25–27	Фиксируем seed=42 для генераторов случайных чисел Python и NumPy
28–29	Создаём папки для кеша и результатов. `/` у Path соединяет части пути; `exist_ok=True` допускает уже существующую папку
30	`setdefault` задаёт кеш Hugging Face только если настройка ещё не задана; `.resolve()` получает абсолютный путь
31–32	Создаём папку ресурсов NLTK и добавляем её первой в список поиска
33–35	Загружаем модели/словари NLTK. Если нужный ресурс недоступен, явно останавливаемся
37–39	Создаём сетевую сессию и политику до трёх повторов временных сбоев. 429 означает слишком много запросов; 5xx — серверные ошибки
40	Пустой словарь журнала загрузок
41–45	Функция SHA-256: читает файл порциями по 1 MiB, обновляет хеш, возвращает строку контрольной суммы. `rb` — чтение байтов
47–51	Начало fetch: создаём родительские папки, проверяем известный хеш уже скачанного файла

Строки	Что происходит
52–58	Если файла нет, скачиваем в `.part`, проверяем HTTP-статус и записываем порциями. `timeout=(20,180)` ограничивает подключение и ожидание чтения, не общее время всей загрузки
59–62	Проверяем хеш новой загрузки, если он задан; при успехе переименовываем временный файл в окончательный
63–65	Не принимаем пустой файл, записываем URL и фактический хеш, возвращаем путь
67–68	Общая функция сохранения CSV. `utf-8-sig` упрощает открытие кириллицы в Excel; `index=False` не пишет служебную нумерацию
70–72	Общая оценка: accuracy и macro F1. Усредняем по истинным классам; `zero_division=0` обрабатывает неопределённый precision/F1
74–78	Строим confusion matrix и показываем график. `fig` — вся картинка, `ax` — область конкретного графика
80–81	Общий стиль графиков и вывод версии Python

Ресурсы NLTK: `punkt_tab` — данные разделения предложений, `wordnet` — словарь лемм, `averaged_perceptron_tagger_eng` — определение частей речи, `stopwords` — стоп-слова, `vader_lexicon` — эмоциональный словарь.

SHA-256 — «отпечаток» содержимого, а не шифрование. Он помогает проверить совпадение с ожидаемым файлом; сам по себе не доказывает, что автор источника заслуживает доверия. У моделей заранее заданы ожидаемые суммы, а у CSV фактические суммы заносятся в журнал. Для CSV без ожидаемого хеша повторный запуск доверяет уже существующему непустому кешу: сохранение суммы в журнал не равно проверке по независимому эталону.

`SEED=42` не делает модель «умнее» и не гарантирует одинаковые вычисления на любом железе. Он фиксирует используемые случайные решения, особенно разбиение и выбор песен.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | from pathlib import Path
002 | from collections import Counter
003 | from functools import lru_cache
004 | import re, json, hashlib, random, html, unicodedata, warnings, importlib.metadata
005 | import numpy as np
006 | import pandas as pd
007 | import matplotlib.pyplot as plt
008 | import requests
009 | from requests.adapters import HTTPAdapter
010 | from urllib3.util.retry import Retry
011 | from IPython.display import display, Markdown
012 | from sklearn.model_selection import train_test_split
013 | from sklearn.feature_extraction.text import TfidfVectorizer
014 | from sklearn.linear_model import LogisticRegression
015 | from sklearn.naive_bayes import MultinomialNB
016 | from sklearn.pipeline import Pipeline
017 | from sklearn.metrics import accuracy_score, f1_score, classification_report, ConfusionMatrixDisplay
018 | from sklearn.metrics.pairwise import cosine_similarity
```

```
import nltk
from nltk.corpus import stopwords, wordnet
from nltk.stem import WordNetLemmatizer
from nltk.sentiment import SentimentIntensityAnalyzer
from wordcloud import WordCloud

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
DATA = Path('nlp_assignment_data'); DATA.mkdir(exist_ok=True)
RESULTS = Path('nlp_assignment_results'); RESULTS.mkdir(exist_ok=True)
os.environ.setdefault('HF_HOME', str((DATA / 'hf_cache').resolve()))
NLTK_DIR = DATA / 'nltk'; NLTK_DIR.mkdir(exist_ok=True)
nltk.data.path.insert(0, str(NLTK_DIR.resolve()))
for resource in ['punkt_tab', 'wordnet', 'averaged_perceptron_tagger_eng', 'stopwords',
'vader_lexicon']:
    if not nltk.download(resource, download_dir=str(NLTK_DIR), quiet=True, raise_on_error=True):
        raise RuntimeError(f'Не удалось загрузить NLTK: {resource}')

session = requests.Session()
session.mount('https://', HTTPAdapter(max_retries=Retry(total=3, backoff_factor=1,
    status_forcelist=[429, 500, 502, 503, 504])))
DOWNLOAD_LOG = {}
def sha256_file(path):
    h = hashlib.sha256()
    with open(path, 'rb') as f:
        for block in iter(lambda: f.read(1024 * 1024), b''): h.update(block)
    return h.hexdigest()

def fetch(url, path, expected_sha256=None):
    """HTTPS + атомарная запись; при заданном хеше проверяется также кеш."""
    path = Path(path); path.parent.mkdir(parents=True, exist_ok=True)
    if path.exists() and expected_sha256 and sha256_file(path) != expected_sha256:
        raise ValueError(f'Контрольная сумма кеша не совпала: {path}. Удалите этот файл и повторите.')
    if not path.exists():
        tmp = path.with_suffix(path.suffix + '.part')
        with session.get(url, stream=True, timeout=(20, 180)) as response:
            response.raise_for_status()
            with open(tmp, 'wb') as f:
                for block in response.iter_content(1024 * 1024):
                    if block: f.write(block)
        if expected_sha256 and sha256_file(tmp) != expected_sha256:
            tmp.unlink(missing_ok=True)
            raise ValueError(f'Контрольная сумма загрузки не совпала: {url}')
        tmp.replace(path)
    if path.stat().st_size == 0: raise ValueError(f'Пустой файл: {path}')
    DOWNLOAD_LOG[str(path)] = {'url': url, 'sha256': sha256_file(path)}
    return path

def save_table(frame, name, index=False):
    frame.to_csv(RESULTS / name, index=index, encoding='utf-8-sig')

def evaluate(y_true, y_pred):
    return {'accuracy': accuracy_score(y_true, y_pred),
            'macro_f1': f1_score(y_true, y_pred, labels=sorted(set(y_true)), average='macro',
zero_division=0)}

def show_confusion(y_true, y_pred, labels, title):
    fig, ax = plt.subplots(figsize=(7, 5))
    ConfusionMatrixDisplay.from_predictions(y_true, y_pred, labels=labels,
        display_labels=labels, cmap='Blues', ax=ax, colorbar=False)
    ax.set_title(title); fig.tight_layout(); plt.show()

plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': False})
print('Python:', sys.version.split()[0])
```

Блок 3. Предобработка английского текста

Ячейка 5 в файле.

Вход: строка отзыва или песни. **Выход:** список лемм либо строка из лемм.

Строки	Объяснение
1	Берём английские стоп-слова и превращаем в множество для быстрой проверки принадлежности
2	Список отрицаний, которые важно сохранить при анализе тональности
3	Создаём лемматизатор WordNet
5	Объявляем функцию; по умолчанию сохраняем отрицания
6	Приводим к строке и нижнему регистру, декодируем HTML (`&` → `&`), унифицируем апостроф
7–9	Раскрываем сокращения: `won't → will not`, `can't → can not`, остальные `n't → not`. Это ограниченный набор правил, не полноценный анализ английской грамматики
10	Удаляем URL и упоминания аккаунтов по регулярному выражению
11	Находим последовательности английских букв: это наша словесная токенизация. Цифры, пунктуация и символы вне a–z исчезают
12	NLTK определяет часть речи каждого слова
13	При `keep_negations=True` вычитаем отрицания из множества удаляемых стоп-слов
14	Переводим начальные буквы POS-тегов NLTK в обозначения WordNet: adjective, verb, noun, adverb
15–16	Оставляем слова длиннее одной буквы и вне стоп-списка, затем лемматизируем. Для неизвестного POS используем noun
18	Декоратор кеширует до 30 000 результатов: повторный одинаковый текст не нужно обрабатывать заново
19–20	Обёртка для отзывов: соединяет список токенов пробелами
22–23	Две проверки: отрицание not не потерялось, eyes превратилось в eye
24–26	Показываем два примера до/после

Пример чтения: `The flights were not good!` → отдельные слова → без служебных слов → лемма `flight`, сохранённое `not`, слово `good`. Точный вывод виден в ячейке. POS важен: `running` как глагол нужно свести к `run`; без POS лемматизатор часто предполагает существительное. Лемматизация отличается от stemming: она пытается получить словарное слово, а stemming обычно обрезает окончания по правилам.

Почему функция не подходит всем языкам? Шаблон `[a-z]+`, английские стоп-слова и WordNet специально рассчитаны на английский. Для задачи 4 предусмотрена другая Unicode-очистка. Двух assert недостаточно, чтобы доказать идеальную обработку всех сокращений и контекстов.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | STOP_EN = set(stopwords.words('english'))
002 | NEGATIONS = {'no', 'not', 'nor', 'never'}
003 | LEMMATIZER = WordNetLemmatizer()
004 |
005 | def english_tokens(text, keep_negations=True):
006 |     text = html.unescape(str(text)).lower().replace('’', "'")
007 |     text = re.sub(r"won't", 'will not', text)
008 |     text = re.sub(r"can't", 'can not', text)
009 |     text = re.sub(r"n't\b", ' not', text)
010 |     text = re.sub(r'https?://\S+|www\.\S+|@\w+', ' ', text)
011 |     words = re.findall(r'[a-z]+', text)
012 |     tagged = nltk.pos_tag(words)
013 |     removed = STOP_EN - NEGATIONS if keep_negations else STOP_EN
014 |     pos_map = {'J': wordnet.ADJ, 'V': wordnet.VERB, 'N': wordnet.NOUN, 'R': wordnet.ADV}
015 |     return [LEMMATIZER.lemmatize(w, pos_map.get(tag[0], wordnet.NOUN))
016 |             for w, tag in tagged if len(w) > 1 and w not in removed]
017 |
018 | @lru_cache(maxsize=30000)
019 | def sentiment_preprocess(text):
020 |     return ' '.join(english_tokens(text, keep_negations=True))
021 |
022 | assert 'not' in english_tokens('The flights were not good!')
023 | assert 'eye' in english_tokens('My eyes and hands')
024 | display(pd.DataFrame({'original': ['The flights were not good!', 'My eyes and hands'],
025 |                       'processed': [sentiment_preprocess(t) for t in
026 |                                     ['The flights were not good!', 'My eyes and hands']]}))
```

Блок 4. Задание 1: загрузка отзывов и разбиение

Ячейка 7 в файле.

Цель — подготовить независимые наборы с правильными метками для сравнения методов.

Строки	Объяснение
1–3	Указываем закреплённый URL, скачиваем CSV через fetch и читаем в pandas
4	Оставляем текст и метку, убираем пропуски, переименовываем метку в label
5–6	Разрешаем ровно три класса и копируем отфильтрованную таблицу
7–8	Создаём processed с очищенными леммами; удаляем строки, где ничего не осталось
9	Для каждого одинакового processed считаем количество разных меток
10	Если одному processed приписаны разные классы, исключаем всю конфликтную группу
11	Удаляем оставшиеся одинаковые processed, перенумеровываем строки
12	Отдаём 40% во временный набор: оставшиеся 60% становятся train

Строки	Объяснение
13	Делим временные 40% пополам: примерно 20% validation, 20% test
14–16	Проверяем, что одинаковые processed не пересекаются между наборами
17–18	Считаем метки по наборам и выводим таблицу распределения
19	Сохраняем очищенную совокупность отзывов

`stratify=reviews.label` сохраняет близкие доли классов в каждом наборе. Это особенно важно, потому что negative заметно больше, чем positive. Из-за целого числа строк доли не обязаны быть математически ровно 60/20/20: получилось 8 388 / 2 796 / 2 797.

Исходный text сохраняется для VADER и чтения человеком, processed — для ML. Два разных исходных текста после удаления слов могут стать одинаковыми; это причина проверять дубликаты именно на используемом моделью представлении. Такая очистка также может удалить содержательно неоднозначные примеры и сделать задачу проще: это ограничение оценки.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | AIRLINE_URL = ('https://raw.githubusercontent.com/martinpella/twitter-airlines/'
002 |     '5ef288ca005842165851e9559fdffcdcfdf7e271/data/Airline-Sentiment-2-w-AA.csv')
003 | reviews = pd.read_csv(fetch(AIRLINE_URL, DATA / 'airline_reviews.csv'))
004 | reviews = reviews[['text', 'airline_sentiment']].dropna().rename(columns={'airline_sentiment':
'label'})
005 | SENTIMENT_LABELS = ['negative', 'neutral', 'positive']
006 | reviews = reviews[reviews.label.isin(SENTIMENT_LABELS)].copy()
007 | reviews['processed'] = reviews.text.map(sentiment_preprocess)
008 | reviews = reviews[reviews.processed.str.len() > 0]
009 | conflicting = reviews.groupby('processed').label.nunique()
010 | reviews = reviews[~reviews.processed.isin(conflicting[conflicting > 1].index)]
011 | reviews = reviews.drop_duplicates('processed').reset_index(drop=True)
012 | sent_train, sent_other = train_test_split(reviews, test_size=.4, random_state=SEED,
stratify=reviews.label)
013 | sent_val, sent_test = train_test_split(sent_other, test_size=.5, random_state=SEED,
stratify=sent_other.label)
014 | assert set(sent_train.processed).isdisjoint(sent_val.processed)
015 | assert set(sent_train.processed).isdisjoint(sent_test.processed)
016 | assert set(sent_val.processed).isdisjoint(sent_test.processed)
017 | display(pd.concat({name: frame.label.value_counts() for name, frame in
018 |     [('train', sent_train), ('validation', sent_val), ('test', sent_test)]},
axis=1).fillna(0).astype(int))
019 | save_table(reviews[['text', 'label']], 'sentiment_clean_data.csv')
```

Блок 5. Задание 1: VADER и выбор порогов

Ячейка 9 в файле.

VADER не обучается на наших отзывах. Его словарь и правила уже готовы; на validation мы настраиваем перевод числовой оценки в один из трёх классов.

Строки	Объяснение
1	Создаём объект словарного анализатора
2–3	Для каждого текста получаем `compound`, складываем числа в NumPy-массив
4–5	Если score < lower — negative; иначе если score > upper — positive; иначе neutral
7	Считаем оценки только для validation
8–11	Перебираем пары порогов и пропускаем невозможные `lower >= upper`
12–13	Переводим оценки в метки, считаем метрики, добавляем пороги и метрики в список
14	Превращаем список в таблицу, сортируем по macro F1. Стабильная сортировка сохраняет порядок при равенстве
15–17	Берём первую, то есть лучшую, пару и показываем результат выбора

На проверенном запуске выбраны **lower=0.0, upper=0.4**. Например, −0.3 → negative, 0.2 → neutral, 0.7 → positive. При score ровно 0 или 0.4 ответ neutral, поскольку сравнения строгие. Compound лежит в [−1,1], но это не вероятность класса positive.

VADER получает исходный текст: регистр, усилители, пунктуация и текстовые эмоциональные сигналы полезны его правилам. Для ML применяется другая предобработка. Это осознанное сравнение двух готовых подходов, а не эксперимент с совершенно одинаковым входным представлением.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | vader = SentimentIntensityAnalyzer()
002 | def lexicon_scores(texts):
003 |     return np.array([vader.polarity_scores(str(t))['compound'] for t in texts])
004 | def classify_scores(scores, lower, upper):
005 |     return np.where(scores < lower, 'negative', np.where(scores > upper, 'positive', 'neutral'))
006 |
007 | val_scores = lexicon_scores(sent_val.text)
008 | threshold_trials = []
009 | for lower in [-.6, -.4, -.2, -.1, -.05, 0.0]:
010 |     for upper in [0.0, .05, .1, .2, .4, .6]:
011 |         if lower >= upper: continue
012 |         pred = classify_scores(val_scores, lower, upper)
013 |         threshold_trials.append({'lower': lower, 'upper': upper, **evaluate(sent_val.label, pred)})
014 | threshold_table = pd.DataFrame(threshold_trials).sort_values('macro_f1', ascending=False,
kind='stable')
015 | best_lower, best_upper = threshold_table.iloc[0][['lower', 'upper']].astype(float)
016 | display(threshold_table.head())
017 | print('Выбранные пороги:', best_lower, best_upper)
```

Блок 6. Задание 1: обучение Logistic Regression и проверка

Ячейка 11 в файле.

Название «регрессия» здесь историческое: Logistic Regression используется как классификатор. Для каждого класса она суммирует веса признаков и преобразует оценки в вероятности. Параметры изучаются по размеченным примерам.

Строки	Объяснение
1–6	Фабрика модели: каждый вызов создаёт новый Pipeline из TF-IDF и LR
3–4	Словесные униграммы и биграммы; min_df=2 — признак минимум в 2 документах; max_df=.98 — не более 98% документов; максимум 35 000 признаков; логарифмический TF
5	C управляет силой регуляризации; balanced увеличивает относительный вес редких классов; max_iter — предел итераций оптимизатора, не число эпох нейросети
8–12	Для C из {0.5,1,2} обучаем отдельный pipeline на train и оцениваем на validation
13–15	Выбираем C с максимальным validation macro F1: в запуске C=2
16–17	Объединяем train и validation; создаём новый итоговый pipeline и обучаем его заново
19–20	Впервые получаем финальные test-предсказания обоих выбранных методов
21–23	Находим самый частый класс обучения и создаём majority baseline — постоянный ответ этим классом
24–28	Считаем и сохраняем общую таблицу test-метрик
29–32	Для каждого подхода выводим precision/recall/F1 по классам и confusion matrix

Что делает fit: TF-IDF изучает словарь и IDF по переданным текстам; LR изучает веса по признакам и меткам. **Что делает predict:** использует уже изученные преобразования и веса. На test словарь не расширяется. При переобучении на train+validation TF-IDF тоже заново обучается на этом расширенном обучающем наборе — это допустимо, test остаётся отдельным.

Регуляризация сдерживает слишком большие веса и помогает бороться с переобучением. В scikit-learn меньше C означает более сильную регуляризацию. `class_weight='balanced'` меняет вклад ошибок классов, но не создаёт новые отзывы и не гарантирует равный recall. Для класса k вес примерно `N / (K × N_k)`.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | def make_sentiment_model(C):
002 |     return Pipeline([
003 |         ('tfidf', TfidfVectorizer(ngram_range=(1, 2), min_df=2, max_df=.98,
004 |                                   max_features=35000, sublinear_tf=True)),
005 |         ('lr', LogisticRegression(C=C, class_weight='balanced', max_iter=1200, random_state=SEED))
006 |     ])
007 |
```

```
008 | ml_trials = []
009 | for C in [.5, 1., 2.]:
010 |     candidate = make_sentiment_model(C)
011 |     candidate.fit(sent_train.processed, sent_train.label)
012 |     ml_trials.append({'C': C, **evaluate(sent_val.label, candidate.predict(sent_val.processed))})
013 | ml_validation = pd.DataFrame(ml_trials).sort_values('macro_f1', ascending=False, kind='stable')
014 | display(ml_validation)
015 | best_C = float(ml_validation.iloc[0].C)
016 | sent_final_train = pd.concat([sent_train, sent_val])
017 | sent_model = make_sentiment_model(best_C).fit(sent_final_train.processed, sent_final_train.label)
018 |
019 | lex_pred = classify_scores(lexicon_scores(sent_test.text), best_lower, best_upper)
020 | ml_pred = sent_model.predict(sent_test.processed)
021 | majority = sent_final_train.label.mode()[0]
022 | sent_metrics = pd.DataFrame([
023 |     {'method': 'Majority baseline', **evaluate(sent_test.label, np.repeat(majority, len(sent_test)))},
024 |     {'method': 'VADER (validation thresholds)', **evaluate(sent_test.label, lex_pred)},
025 |     {'method': 'TF-IDF + Logistic Regression', **evaluate(sent_test.label, ml_pred)}
026 | ])
027 | display(sent_metrics)
028 | save_table(sent_metrics, 'task1_metrics.csv')
029 | for name, pred in [('VADER', lex_pred), ('TF-IDF + LR', ml_pred)]:
030 |     print(name)
031 |     print(classification_report(sent_test.label, pred, labels=SENTIMENT_LABELS, zero_division=0))
032 |     show_confusion(sent_test.label, pred, SENTIMENT_LABELS, name)
```

Блок 7. Задание 1: разбор ошибок и функция для нового отзыва

Ячейка 13 в файле.

Строки	Объяснение
1–3	Для каждого метода добавляем predicted и оставляем строки, где он не совпал с label
4	Читаем первые 8 ошибок. Это иллюстративные примеры, не случайная репрезентативная выборка всех ошибок
5–10	Проверяем признаки возможных причин: отрицание, положительные слова в жалобе, недостаток контекста
11–15	Сохраняем исходный текст, истинную метку, предсказание и гипотезу; выводим и записываем CSV
17–21	Функция predict_sentiment одновременно возвращает compound VADER и ответы двух методов для одного нового текста
22–24	Проверяем положительный, отрицательный, нейтральный и потенциально ироничный примеры

`[text]` нужен потому, что predict ожидает коллекцию документов; `[0]` достаёт единственный ответ. Автоматическая подсказка «проверить иронию» не доказывает, что текст саркастический.

Показательный пример: `Great, another three-hour delay.` VADER дал positive, а ML — negative. Слово great положительное, но в контексте очередной задержки звучит иронично. Один удачный ответ ML не означает, что модель научилась понимать любой сарказм.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | error_rows = []
002 | for method, predictions in [('VADER', lex_pred), ('TF-IDF + LR', ml_pred)]:
003 |     errors = sent_test.assign(predicted=predictions).query('label != predicted')
004 |     for row in errors.head(8).itertuples():
005 |         lower = row.text.lower()
006 |         hints = []
007 |         if re.search(r"\b(not|no|never)\b|n't", lower): hints.append('проверить область действия
отрицания')
008 |         if any(w in lower for w in ['thanks', 'great', 'love', 'awesome']):
009 |             hints.append('проверить иронию / смешанную тональность; возможна неоднозначность разметки')
010 |         if not hints: hints.append('проверить контекст услуги, многозначность и потери при
предобработке')
011 |         error_rows.append({'method': method, 'text': row.text, 'true': row.label,
012 |                            'predicted': row.predicted, 'hypothesis_to_check': '; '.join(hints)})
013 | sentiment_errors = pd.DataFrame(error_rows)
014 | with pd.option_context('display.max_colwidth', 180): display(sentiment_errors)
015 | save_table(sentiment_errors, 'task1_errors.csv')
016 |
017 | def predict_sentiment(text):
018 |     score = float(lexicon_scores([text])[0])
019 |     return {'text': text, 'lexicon_score': score,
020 |         'VADER': classify_scores(np.array([score]), best_lower, best_upper)[0],
021 |         'ML': sent_model.predict([sentiment_preprocess(text)])[0]}
022 | display(pd.DataFrame([predict_sentiment(t) for t in [
023 |     'I loved the friendly service!', 'My flight was cancelled and nobody helped.',
024 |     'The flight departs at six.', 'Great, another three-hour delay.']]))
```

Блок 8. Задание 1: облака слов и важные признаки

Ячейка 14 в файле.

Строки	Объяснение
1–4	Используем только train+validation; отдельно собираем токены positive и negative, считаем Counter
5–8	Создаём облака: размер слова зависит от частоты. `collocations=False` отключает объединение слов в пары самой библиотекой; сохраняем PNG
10–11	Получаем названия признаков TF-IDF и объект обученного классификатора через named_steps
12–14	Создаём три графика: по одному на класс; связываем метку с её строкой коэффициентов
15	argsort возвращает индексы весов по возрастанию; `[-12:]` выбирает 12 самых больших
16–19	Рисуем горизонтальные столбцы и сохраняем признаки с весами
20–23	Сравниваем два содержательных метода по test macro F1 и формируем текстовый вывод

Частота ≠ важность для классификации. Слово может быть частым в обоих классах, но почти не помогать их различать. Большой положительный коэффициент повышает линейную оценку соответствующего класса при прочих равных. Это ассоциация в обучающих данных, не доказательство причины. Часть признаков — биграммы: они содержат два слова. Облако не является метрикой качества.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | # Облака слов строим по обучающим данным, не по тесту.
002 | fig, axes = plt.subplots(1, 2, figsize=(14, 5))
003 | for ax, label in zip(axes, ['positive', 'negative']):
004 |     frequencies = Counter(' '.join(sent_final_train.loc[sent_final_train.label == label,
'processed']).split())
005 |     cloud = WordCloud(width=800, height=450, background_color='white', random_state=SEED,
006 |                       collocations=False).generate_from_frequencies(frequencies)
007 |     ax.imshow(cloud, interpolation='bilinear'); ax.axis('off'); ax.set_title(label)
008 | fig.tight_layout(); fig.savefig(RESULTS / 'task1_wordclouds.png', dpi=150); plt.show()
009 |
010 | features = sent_model.named_steps['tfidf'].get_feature_names_out()
011 | classifier = sent_model.named_steps['lr']
012 | fig, axes = plt.subplots(1, 3, figsize=(17, 5))
013 | weight_rows = []
014 | for ax, label, weights in zip(axes, classifier.classes_, classifier.coef_):
015 |     top = np.argsort(weights)[-12:]
016 |     ax.barh(features[top], weights[top]); ax.set_title(f'Weights for {label}')
017 |     for j in top: weight_rows.append({'class': label, 'feature': features[j], 'weight': weights[j]})
018 | fig.tight_layout(); fig.savefig(RESULTS / 'task1_feature_weights.png', dpi=150); plt.show()
019 | save_table(pd.DataFrame(weight_rows), 'task1_top_features.csv')
020 | best = sent_metrics.iloc[1:].sort_values('macro_f1', ascending=False).iloc[0]
021 | display(Markdown(f'**Вывод:** на этом test лучше по macro F1 метод **{best.method}** '
022 |     f'({best.macro_f1:.3f}). Для улучшения: учитывать область отрицания, словарь авиаперевозок, '
023 |     'контекстные модели и отдельно оценивать сарказм. Перенос на другие домены требует новой
проверки.'))
```

Блок 9. Задание 2: встроенные статьи

Ячейка 16 в файле.

В этой ячейке одна переменная `EMBEDDED_ARTICLES` — словарь из пяти пар: имя файла → полный исходный текст. Строковые литералы здесь являются данными. Внутренние `\n` обозначают переносы строк. Мы не запускаем содержимое статей как Python.

- article1 — извлечение мелодии из PCM, перевод в MIDI и сравнение последовательностей.
- article2 — сходство мелодий, alignment/shifting, индексирование и быстрый поиск.
- article3 — аудиоотпечатки, MFCC, энтропия и энергия; в тексте повторяется abstract.
- article4 — идентификация музыки, сэмплирование и отпечатки.
- article5 — ритм, Amen Break и мелодическое заимствование.

Встраивание нужно, чтобы ноутбук работал без повторной загрузки пяти приложений. Это не интернет-датасет и не заранее заданная сводка. Настоящая обработка выполняется ниже.

Данные этой ячейки:

Ключ	Символов в исходном тексте
`article1.txt`	2814
`article2.txt`	4450
`article3.txt`	4002
`article4.txt`	3401
`article5.txt`	2009

Блок 10. Задание 2: очистка статей и экстрактивная сводка

Ячейка 17 в файле.

Экстрактивный метод выбирает целые исходные предложения. Нового текста модель здесь не пишет.

Строки	Объяснение
1–6	NFKC унифицирует совместимые Unicode-формы; удаляем управляющий символ, переносы слов через дефис, некоторые заголовки и ссылки вида [12]; сжимаем пробелы
8–12	Для каждого имени читаем локальный файл при наличии, иначе берём встроенный текст; чистим и сохраняем в documents
13–14	Показываем размер документов: слова по split и предложения по sent_tokenize
16	Берём первые три документа: article1, article2, article3. Порядок словаря сохраняется Python
17–22	Разбиваем на предложения, отбрасываем короче 5 слов, удаляем точные повторы, записываем документ, позицию и текст
23–26	Создаём таблицу; sentence_key игнорирует регистр и пунктуацию; убираем такие повторы внутри документа
28–29	Строим одно общее TF-IDF-пространство предложений трёх статей
30	Средний вектор всех предложений — центроид, приблизительное представление общей темы
31	Косинусное сходство каждого предложения с центроидом становится его score
33–38	В каждом документе берём 3 максимальных score, затем восстанавливаем их исходный порядок
42–47	Сортируем весь общий набор по score; удаляем совпадающие sentence_key между документами; берём 6 лучших уникальных предложений и выводим с источниками
48–50	Сохраняем все оценки, общий top-6 и отдельные сводки документов

Строки 40–41 — комментарии. Точнее описывать результат как **шесть самых высоко оценённых уникальных нормализованных предложений**: удаление точных нормализованных повторов есть, но квоты по документам и MMR для смысловых повторов не применяются.

`axis=0` усредняет строки матрицы по каждому признаку; `.ravel()` превращает матрицу оценок в одномерный массив. Косинусное сходство — `a·b / (||a|| × ||b||)`: смотрим на направление векторов, а не только на их длину. Векторы TF-IDF неотрицательны, поэтому здесь оценки находятся примерно в диапазоне 0–1.

Центроид не понимает научную ценность: общая лексика music/plagiarism/melody может поднять даже фразу «в разделе 5 описаны эксперименты». В выводе это действительно видно. Точный повтор убирается, а две почти одинаковые по смыслу фразы могут остаться.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | def clean_article(text):
002 |     text = unicodedata.normalize('NFKC', text).replace('\x02', '')
003 |     text = re.sub(r'(?<=\w)-\s*\n\s*(?=\w)', '', text)
004 |     text = re.sub(r'^\s*(?:\d+\.\s*)?(?:Introduction|Rhythm Plagiarism)\s*$', '', text, flags=re.M)
005 |     text = re.sub(r'\[\d+\]', '', text)
006 |     return re.sub(r'\s+', ' ', text).strip()
007 |
008 | documents = {}
009 | for name, embedded in EMBEDDED_ARTICLES.items():
010 |     local_path = Path(name)
011 |     original = local_path.read_text(encoding='utf-8-sig') if local_path.exists() else embedded
012 |     documents[name] = clean_article(original)
013 | display(pd.DataFrame([{'document': name, 'words': len(text.split()),
014 |     'sentences': len(nltk.sent_tokenize(text))} for name, text in documents.items()()]))
015 |
016 | extractive_docs = list(documents)[:3]
017 | sentence_rows = []
018 | for doc in extractive_docs:
019 |     sentences = list(dict.fromkeys(s.strip() for s in nltk.sent_tokenize(documents[doc]) if
len(s.split()) >= 5))
020 |     assert len(sentences) >= 3, f'Слишком мало предложений: {doc}'
021 |     sentence_rows.extend({'document': doc, 'position': i, 'sentence': sentence}
022 |                          for i, sentence in enumerate(sentences))
023 | sentence_frame = pd.DataFrame(sentence_rows)
024 | sentence_frame['sentence_key'] = sentence_frame.sentence.map(
025 |     lambda s: re.sub(r'[^a-z0-9]+', ' ', s.lower()).strip())
026 | sentence_frame = sentence_frame.drop_duplicates(['document', 'sentence_key']).reset_index(drop=True)
027 |
028 | sentence_vectorizer = TfidfVectorizer(stop_words='english', ngram_range=(1, 2), sublinear_tf=True)
029 | sentence_X = sentence_vectorizer.fit_transform(sentence_frame.sentence)
030 | centroid = np.asarray(sentence_X.mean(axis=0))
031 | sentence_frame['score'] = cosine_similarity(sentence_X, centroid).ravel()
032 |
033 | per_document = {}
034 | for name in extractive_docs:
035 |     top = sentence_frame[sentence_frame.document == name].nlargest(3, 'score').sort_values('position')
036 |     per_document[name] = top.sentence.tolist()
037 |     display(Markdown(f'### {name}: 3 ключевых предложения'))
038 |     for i, sentence in enumerate(top.sentence, 1): print(f'{i}. {sentence}')
039 |
040 | # Здесь именно 6 максимальных весов из общего набора, как требуется в PDF.
041 | # Квоты по документам и подавление похожих предложений не меняют основной результат.
042 | combined_six = (sentence_frame.sort_values('score', ascending=False, kind='stable')
043 |     .drop_duplicates('sentence_key').head(6).reset_index(drop=True))
044 | assert len(combined_six) == 6
```

```
045 | display(Markdown('### Единый список: 6 предложений по убыванию веса'))
046 | for i, row in enumerate(combined_six.itertuples(), 1):
047 |     print(f'{i}. [{row.document}; score={row.score:.3f}] {row.sentence}')
048 | save_table(sentence_frame, 'task2_sentence_scores.csv')
049 | save_table(combined_six, 'task2_combined_six.csv')
050 | (RESULTS / 'task2_extractive.json').write_text(json.dumps(per_document, ensure_ascii=False, indent=2),
encoding='utf-8')
```

Блок 11. Задание 2: контрольные суммы T5

Ячейка 19 в файле.

`T5_CHECKSUMS` — словарь `путь файла → ожидаемый SHA-256`. Конфиги задают архитектуру и генерацию; tokenizer/spiece — преобразование текста в токены; два ONNX-файла содержат вычислительные графы и параметры encoder/decoder.

У каждого файла своя сумма. Следующий блок вызывает fetch с ней, чтобы не принять повреждённую или отличающуюся копию. Длинные шестнадцатеричные строки не нужно запоминать на защите: нужно понимать назначение. Проверка хеша не оценивает языковое качество модели.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | T5_CHECKSUMS = {
002 |   "config.json": "8965dc57868800ffb28ecec5b67c1f751384995cecc800f7616987ecf7b69f71",
003 |   "generation_config.json": "05c4044888661843888 96de17869dee9fc21b2fe10f77512c7aa21f51f0a6e65",
004 |   "special_tokens_map.json": "5c87151ef0f72a99d1f766a4c418bd2a1f90aaa30a8e22fe5eca9641daebb64f",
005 |   "spiece.model": "d60acb128cf7b7f2536e8f38a5b18a05535c9e14c7a355904270e15b0945ea86",
006 |   "tokenizer.json": "5f0ed8ab5b8cfa9812bb73752f1d80c292e52bcf5a87a144dc9ab2d251056cbb",
007 |   "tokenizer_config.json": "d1e7146101aa96282057f374aa9c3b260fba2109e5990edb1775d6efc70ffa3c",
008 |   "onnx/encoder_model_quantized.onnx":
"966c5cb80c2805821aa9eac5fb26dcaad44de0fee11b990ffc97500ae83044a7",
009 |   "onnx/decoder_model_merged_quantized.onnx":
"022ea05ec1c067113e03e4513c7a4c498044b316959de0a7c1dd1950e3308aa6"
010 | }
```

Блок 12. Задание 2: T5, токены и иерархическая генерация

Ячейка 20 в файле.

Предобученная T5-small преобразует входную последовательность в выходную. Encoder кодирует исходный текст; decoder последовательно генерирует токены сводки. Механизм attention позволяет связывать части входа с текущим выходом. Мы используем готовые веса.

Строки	Объяснение
1–4	Загружаем PyTorch, ONNX Runtime, токенизатор и обёртку модели
6–9	Скачиваем закреплённую ONNX-копию T5-small, проверяя каждую сумму
11	Загружаем токенизатор только с диска, не обращаясь дополнительно к Hugging Face

Строки	Объяснение
12–14	Ограничиваем CPU-параллелизм максимум 4 потоками внутри операции и 1 между операциями
15–19	Загружаем encoder и объединённый decoder на CPU. use_cache переиспользует вычисления внимания; это другой кеш, чем файлы загрузок
21	Считаем токены без служебных маркеров
23–26	Кодируем длинный текст в IDs, режем по 430 токенов и декодируем каждый кусок обратно в строку
28–30	Добавляем префикс задачи `summarize:`, получаем PyTorch-тензоры, проверяем ограничение входа 512. truncation=False запрещает незаметно обрезать вход
31–35	Генерируем без градиентов; возвращаем обычный текст без служебных токенов
37–47	Проходим все 5 документов и все их куски; получаем краткие описания и журнал источников
49–55	Склеиваем промежуточные сводки; если они длиннее 430 токенов, повторно сжимаем порциями. Не более 8 уровней; отсутствие сокращения вызывает ошибку
56–58	Убеждаемся, что вход помещается; генерируем окончательную сводку до 150 новых токенов; проверяем непустоту и меньшую длину, чем у корпуса
59–62	Показываем текст и сохраняем итог с промежуточными результатами

Токен не обязательно слово. Редкое слово может распасться на несколько частей; пунктуация тоже может иметь токены. 430 оставляет запас под префикс, специальные маркеры и возможные изменения при decode/encode. Разрезание по токенам может оборвать предложение: это недостаток простого chunking, который можно улучшить границами предложений.

Параметры generate:

- max_new_tokens — верхняя граница числа новых токенов, не количество слов.
- min_new_tokens=20 — не заканчивать слишком рано; это не требование ровно 20 слов.
- num_beams=4 — beam search сохраняет несколько перспективных продолжений.
- do_sample=False — не использовать случайную выборку следующего токена.
- no_repeat_ngram_size=3 — не повторять одну и ту же выходную тройку токенов.
- repetition_penalty=1.1 — дополнительно штрафовать повторение токенов.
- early_stopping=True — завершать поиск по правилам beam search, когда готово достаточно завершённых кандидатов.

INT8-квантование экономит память и меняет точность вычислений. Мы не измеряли отдельно потерю качества от INT8 относительно исходной T5, поэтому нельзя утверждать, что все ошибки вызваны именно квантованием. Число параметров сети в 8 раз не уменьшается: меняется формат хранения/вычисления части весов. ONNX — формат и среда выполнения, не новая архитектура.

В выводе есть предупреждение о длинной последовательности при **подсчёте и нарезке** целого текста через tokenizer.encode. Перед реальным запуском модели стоит assert длины ≤512, и проверенный запуск завершился. Это не означает, что модель получила весь длинный документ за один проход. Предупреждение IProgress относится к виджету прогресса, не к качеству сводки.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | import torch
002 | import onnxruntime as ort
003 | from transformers import AutoTokenizer
004 | from optimum.onnxruntime import ORTModelForSeq2SeqLM
005 |
006 | T5_BASE = 'https://raw.githubusercontent.com/Arfius/t5-small/88f269658b785fecf015c2f787a3fa9486d349f4/'
007 | T5_DIR = DATA / 't5_small_onnx'
008 | for filename, digest in T5_CHECKSUMS.items():
009 |     fetch(T5_BASE + filename, T5_DIR / filename, expected_sha256=digest)
010 |
011 | tokenizer = AutoTokenizer.from_pretrained(str(T5_DIR), local_files_only=True)
012 | ort_options = ort.SessionOptions()
013 | ort_options.intra_op_num_threads = min(4, os.cpu_count() or 1)
014 | ort_options.inter_op_num_threads = 1
015 | summary_model = ORTModelForSeq2SeqLM.from_pretrained(str(T5_DIR),
016 |     encoder_file_name='onnx/encoder_model_quantized.onnx',
017 |     decoder_file_name='onnx/decoder_model_merged_quantized.onnx',
018 |     use_merged=True, use_cache=True, local_files_only=True,
019 |     session_options=ort_options, provider='CPUExecutionProvider')
020 |
021 | def token_count(text): return len(tokenizer.encode(text, add_special_tokens=False))
022 |
023 | def token_chunks(text, size=430):
024 |     ids = tokenizer.encode(text, add_special_tokens=False)
025 |     return [tokenizer.decode(ids[i:i+size], skip_special_tokens=True)
026 |             for i in range(0, len(ids), size)]
027 |
028 | def summarize_chunk(text, max_new_tokens=85):
029 |     inputs = tokenizer('summarize: ' + text, return_tensors='pt', truncation=False)
030 |     assert inputs['input_ids'].shape[1] <= 512, 'Ошибка разбиения на токеновые блоки'
031 |     with torch.inference_mode():
032 |         generated = summary_model.generate(**inputs, max_new_tokens=max_new_tokens,
033 |             min_new_tokens=20, num_beams=4, do_sample=False,
034 |             no_repeat_ngram_size=3, repetition_penalty=1.1, early_stopping=True)
035 |     return tokenizer.decode(generated[0], skip_special_tokens=True).strip()
036 |
037 | document_digests = {}
038 | chunk_log = []
039 | for name, text in documents.items():
040 |     chunks = token_chunks(text)
041 |     summaries = []
042 |     for i, chunk in enumerate(chunks):
043 |         summary = summarize_chunk(chunk, max_new_tokens=75)
044 |         summaries.append(summary)
045 |         chunk_log.append({'document': name, 'chunk': i, 'tokens': token_count(chunk), 'summary':
summary})
046 |     document_digests[name] = ' '.join(summaries)
047 |     print(name, '→', len(chunks), 'блоков обработано')
048 |
049 | merged = ' '.join(document_digests.values())
050 | for _ in range(8):
051 |     if token_count(merged) <= 430: break
052 |     smaller = ' '.join(summarize_chunk(c, max_new_tokens=65) for c in token_chunks(merged))
053 |     if token_count(smaller) >= token_count(merged):
054 |         raise RuntimeError('Сводка не сокращается: необходимо уменьшить max_new_tokens.')
```

```
055 |     merged = smaller
056 | assert token_count(merged) <= 430
057 | abstractive_summary = summarize_chunk(merged, max_new_tokens=150)
058 | assert abstractive_summary and len(abstractive_summary.split()) < sum(len(t.split()) for t in
documents.values())
059 | display(Markdown('### Сгенерированная абстрактивная сводка (английский)'))
060 | print(abstractive_summary)
061 | save_table(pd.DataFrame(chunk_log), 'task2_chunk_summaries.csv')
062 | (RESULTS / 'task2_abstractive.txt').write_text(abstractive_summary, encoding='utf-8')
```

Блок 13. Задание 2: ROUGE, покрытие и проверка опорных предложений

Ячейка 22 в файле.

Здесь оцениваем уже полученные сводки; новая модель не обучается.

Строки	Объяснение
1	Импортируем оценщик ROUGE
2–20	Две авторские учебные эталонные сводки: для 3 и для 5 статей; соседние строковые литералы в скобках Python склеивает автоматически
21	Соединяем общий extractive top-6 в один текст
22	Создаём ROUGE-1, ROUGE-2, ROUGE-L; stemming смягчает различия словоформ при оценивании
23–32	Для каждого метода считаем ROUGE, слова и отношение длины к длине соответствующих исходников; сохраняем таблицу
34–43	Сравниваем TF-IDF-вектор сводки с каждым её исходным документом, получаем лексический прокси покрытия
45–50	Собираем исходные предложения всех 5 статей, предложения генерации и общее пространство признаков
51–59	Для каждого сгенерированного предложения находим наиболее похожее исходное и выводим его как материал для ручной проверки

`scorer.score(reference, summary)` получает сначала эталон, потом кандидата. `fmeasure` — F1 соответствующего ROUGE. ROUGE-1 смотрит отдельные слова, ROUGE-2 — соседние пары, ROUGE-L — наибольшую общую подпоследовательность с сохранением порядка. Она не обязана быть непрерывным фрагментом.

`compression_ratio = слова_сводки / слова_источников`. Значение 0.1 означает примерно 10% исходной длины. Меньше не обязательно лучше: можно потерять почти все важные факты.

Эталоны составлены в рамках решения, а не независимой экспертной комиссией. Входы extractive и abstractive различаются (3 против 5 статей), поэтому их ROUGE нельзя читать как честный прямой турнир. Косинусное сходство помогает искать опоры, но похожая лексика не доказывает истинность: «метод работает» и «метод не работает» могут быть близки.

После этого блока идут Markdown-разделы с фактической критикой T5 и **отдельной авторской редактурой аннотации**. Эта редактура не является выводом generate и не должна так называться на защите. Последнее число в выводе некоторых ячеек — возвращаемое write_text количество символов, а не качество, не число слов и не число ошибок.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | from rouge_score import rouge_scorer
002 | REFERENCE_THREE = (
003 |     'The articles address automatic detection of music plagiarism through melody similarity and audio
fingerprints. '
004 |     'Melody is a distinctive musical element, and similar melodic sequences can indicate possible
copying. '
005 |     'Content-based indexing and query processing help search large music databases. '
006 |     'One system extracts melody from polyphonic audio and converts it to MIDI notes. '
007 |     'Another uses MFCC, entropy and energy with vector quantization and Euclidean comparison. '
008 |     'Quantitative comparison can support more reliable assessment than subjective listening alone, '
009 |     'but detecting similarity does not by itself establish legal plagiarism.')
010 | REFERENCE_FIVE = (
011 |     'Music plagiarism detection requires scalable methods to compare musical works. '
012 |     'The documents focus on melody similarity, content-based retrieval and quantitative analysis '
013 |     'to support human judgments about possible copying. '
014 |     'Digital music collections create a need for efficient indexing and automatic identification. '
015 |     'Methods include PCM-to-MIDI melody extraction, alignment and shifting, '
016 |     'multidimensional indexing, and fingerprints using MFCC, entropy and energy. '
017 |     'Sampling can reuse and transform recorded excerpts, requiring audio matching. '
018 |     'Rhythm is another relevant dimension; the Amen Break illustrates widely reused drum patterns. '
019 |     'Rhythmic similarity involves temporal patterns and amplitude envelopes and is less studied than
melody. '
020 |     'Automatic similarity scores assist investigation but cannot alone determine copyright
infringement.')
021 | extractive_text = ' '.join(combined_six.sentence)
022 | scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'], use_stemmer=True)
023 | quality_rows = []
024 | for method, summary, reference, names in [
025 |     ('Extractive (3 documents)', extractive_text, REFERENCE_THREE, extractive_docs),
026 |     ('Abstractive (5 documents)', abstractive_summary, REFERENCE_FIVE, list(documents))]:
027 |     scores = scorer.score(reference, summary)
028 |     quality_rows.append({'method': method, 'words': len(summary.split()),
029 |         'compression_ratio': len(summary.split()) / sum(len(documents[n].split()) for n in names),
030 |         **{f'{k}_F1': v.fmeasure for k, v in scores.items()}})
031 | quality = pd.DataFrame(quality_rows)
032 | display(quality); save_table(quality, 'task2_quality_metrics.csv')
033 |
034 | coverage_rows = []
035 | for method, summary in [('extractive_3_docs', extractive_text), ('abstractive_5_docs',
abstractive_summary)]:
036 |     names = extractive_docs if method.startswith('extractive') else list(documents)
037 |     vec = TfidfVectorizer(stop_words='english')
038 |     matrix = vec.fit_transform([documents[n] for n in names] + [summary])
039 |     similarities = cosine_similarity(matrix[:-1], matrix[-1]).ravel()
040 |     coverage_rows.extend({'method': method, 'document': n, 'lexical_coverage_proxy': float(v)}
041 |                          for n, v in zip(names, similarities))
042 | coverage = pd.DataFrame(coverage_rows)
043 | display(coverage); save_table(coverage, 'task2_document_coverage.csv')
044 |
045 | source_sentences = [(name, sentence) for name, text in documents.items()
046 |                     for sentence in nltk.sent_tokenize(text) if len(sentence.split()) >= 5]
047 | generated_sentences = nltk.sent_tokenize(abstractive_summary)
048 | ground_vec = TfidfVectorizer(stop_words='english').fit(
049 |     [s for _, s in source_sentences] + generated_sentences)
050 | source_X = ground_vec.transform([s for _, s in source_sentences])
```

```
051 | support_rows = []
052 | for sentence in generated_sentences:
053 |     similarity = cosine_similarity(ground_vec.transform([sentence]), source_X).ravel()
054 |     best = int(similarity.argmax())
055 |     support_rows.append({'generated': sentence, 'closest_source': source_sentences[best][0],
056 |         'source_sentence': source_sentences[best][1], 'similarity_proxy': float(similarity[best])})
057 | support = pd.DataFrame(support_rows)
058 | with pd.option_context('display.max_colwidth', 220): display(support)
059 | save_table(support, 'task2_factual_support_review.csv')
```

Блок 14. Задание 3: выбор 40 песен и top-10

Ячейка 26 в файле.

В этой задаче нет target для обучения. Жанр нужен, чтобы группировать тексты и сравнивать частоты.

Строки	Объяснение
1–4	Загружаем CSV с закреплённого коммита; упрощаем названия колонок до genre и lyrics
5	Сохраняем порядковый номер строки исходной таблицы через np.arange
6–8	Убираем пропуски, создаём нормализованный ключ, оставляем тексты минимум из 30 слов и убираем повторы
9–12	Показываем доступные жанры, выбираем четыре и проверяем наличие минимум 10 песен каждого
13–14	Для каждого жанра случайно выбираем 10 строк с фиксированным seed и объединяем
15	Предобрабатываем тексты общей английской функцией; здесь отрицания не сохраняем отдельно
16–19	Создаём хеши, проверяем 40 уникальных текстов и по 10 на жанр; сохраняем идентификаторы
20–21	В каждом жанре объединяем все списки токенов и считаем Counter
22–24	most_common(10) возвращает десять наиболее частых слов; превращаем результат в таблицу и сохраняем
25–29	Четыре горизонтальных графика. `[::-1]` разворачивает top-10 для удобного порядка по вертикали

Counter превращает `['love','day','love']` в соответствие `love:2, day:1`. Вложенное `for words ... for w in words` читает сначала список токенов одной песни, потом каждый токен этого списка. Это обычное «расплющивание» нескольких списков.

Корпус уже очищен авторами: встречаются `im`, `dont`, `cant`. Наше правило для `can't` не восстановит апостроф в `cant`, а WordNet не обязан исправлять интернет-сокращения. Поэтому эти формы попали в top-10 — реальное ограничение предобработки, а не ошибка Counter. Улучшение — доменный словарь сокращений и явный эксперимент до/после, а не ручное удаление неудобных результатов из отчёта.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | LYRICS_URL = ('https://raw.githubusercontent.com/etarakci/music-genre-prediction/'
002 |     'cd007c16892ff98b40f8df1e33d26e3c8cf80127/data/genre_lyrics_data.csv')
003 | lyrics_all = pd.read_csv(fetch(LYRICS_URL, DATA / 'genre_lyrics_data.csv'))
004 | lyrics_all = lyrics_all.rename(columns={'genre_name': 'genre', 'lyrics_cleaned': 'lyrics'})
005 | lyrics_all['source_row'] = np.arange(len(lyrics_all))
006 | lyrics_all = lyrics_all.dropna(subset=['genre', 'lyrics']).copy()
007 | lyrics_all['canonical'] = lyrics_all.lyrics.map(lambda t: re.sub(r'[^a-z]+', ' ',
str(t).lower()).strip())
008 | lyrics_all = lyrics_all[lyrics_all.canonical.str.split().str.len() >= 30].drop_duplicates('canonical')
009 | display(lyrics_all.genre.value_counts().head(20).to_frame('available_songs'))
010 | GENRES = ['country', 'blues', 'folk', 'hip-hop']
011 | missing = [g for g in GENRES if (lyrics_all.genre == g).sum() < 10]
012 | assert not missing, f'Не хватает 10 уникальных песен в жанрах {missing}. Доступные:
{lyrics_all.genre.unique()}'
013 | lyrics_sample = pd.concat([lyrics_all[lyrics_all.genre == g].sample(10, random_state=SEED)
014 |                            for g in GENRES], ignore_index=True)
015 | lyrics_sample['tokens'] = lyrics_sample.lyrics.map(lambda t: english_tokens(t, keep_negations=False))
016 | lyrics_sample['text_sha256'] = lyrics_sample.canonical.map(lambda t:
hashlib.sha256(t.encode()).hexdigest())
017 | assert len(lyrics_sample) == 40 and lyrics_sample.text_sha256.nunique() == 40
018 | assert (lyrics_sample.groupby('genre').size() == 10).all()
019 | save_table(lyrics_sample[['source_row', 'genre', 'text_sha256']], 'task3_selected_songs.csv')
020 | genre_counts = {genre: Counter(w for words in lyrics_sample.loc[lyrics_sample.genre == genre, 'tokens']
021 |                                for w in words) for genre in GENRES}
022 | top10 = pd.DataFrame([{'genre': genre, 'word': word, 'count': count}
023 |     for genre, counts in genre_counts.items() for word, count in counts.most_common(10)])
024 | display(top10); save_table(top10, 'task3_top10.csv')
025 | fig, axes = plt.subplots(2, 2, figsize=(13, 9))
026 | for ax, genre in zip(axes.flat, GENRES):
027 |     top = genre_counts[genre].most_common(10)[::-1]
028 |     ax.barh([w for w, _ in top], [c for _, c in top]); ax.set_title(genre)
029 | fig.tight_layout(); fig.savefig(RESULTS / 'task3_top10.png', dpi=150); plt.show()
```

Блок 15. Задание 3: категории и два определения редкости

Ячейка 27 в файле.

Строки	Объяснение
1–7	Словарь тематических категорий: значения — множества слов в форме лемм
8–15	Для каждого жанра и каждого слова категории берём частоту; делим её на все обработанные токены жанра и умножаем на 1000
16–19	Создаём таблицу; в выводе отдельных слов скрываем нули; суммируем слова по категории и жанру
20–24	Строим диаграмму относительных частот и сохраняем обе таблицы
27–32	Первый вариант редкости: слово встречается в данном жанре от 1 до 3 раз; полный список сохраняем, на экран выводим первые 15 в группе
34	Пересекаем множества словарей всех жанров: остаются только слова, присутствующие в каждом

Строки	Объяснение
35–41	Второй вариант: среди общих слов оставляем те, у которых в КАЖДОМ жанре от 1 до 3 вхождений. Сохраняем все найденные
43–47	Для каждого жанра находим наибольшую из заданных тематических категорий и печатаем краткий вывод

Две трактовки показаны из-за неоднозначности PDF. Это не одно и то же: слово с частотами `1,0,2,1` редко в трёх жанрах, но отсутствует в одном и не подходит во вторую таблицу. Слово с частотами `3,3,3,3` подходит во вторую, хотя в общем корпусе у него 12 вхождений.

Пример нормирования: 10 упоминаний на 1000 токенов и 10 на 2000 — одинаковое абсолютное число, но относительные частоты 10 и 5 на тысячу. Знаменатель в коде — **все оставшиеся после обработки токены жанра**, а не число песен и не число только тематических слов.

Словарь не различает смыслы: `heart` может быть метафорой, `fall` — глаголом «падать». Нулевая частота слова из словаря — измеренное отсутствие упоминания в выборке, не доказательство, что тема никогда не встречается в жанре.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | # Категории заданы в лемматизированном виде: eyes -> eye, hands -> hand.
002 | CATEGORIES = {
003 |     'time_of_day': {'morning', 'day', 'afternoon', 'evening', 'night', 'midnight', 'dawn', 'dusk'},
004 |     'seasons': {'winter', 'spring', 'summer', 'autumn', 'fall'},
005 |     'body_parts': {'eye', 'heart', 'hand', 'head', 'face', 'arm', 'leg', 'foot', 'hair',
006 |                    'mouth', 'lip', 'ear', 'finger', 'skin', 'body', 'blood', 'bone'}
007 | }
008 | category_rows = []
009 | for genre, counts in genre_counts.items():
010 |     total = sum(counts.values())
011 |     for category, dictionary in CATEGORIES.items():
012 |         for word in sorted(dictionary):
013 |             n = counts[word]
014 |             category_rows.append({'genre': genre, 'category': category, 'word': word,
015 |                                   'count': n, 'per_1000_tokens': 1000*n/total})
016 | category_words = pd.DataFrame(category_rows)
017 | display(category_words.query('count > 0').sort_values(['genre', 'category', 'count'], ascending=[True,
True, False]))
018 | category_totals = category_words.groupby(['genre', 'category'])[['count',
'per_1000_tokens']].sum().reset_index()
019 | display(category_totals)
020 | category_totals.pivot(index='genre', columns='category',
values='per_1000_tokens').plot.bar(figsize=(11, 4))
021 | plt.ylabel('Mentions per 1,000 processed tokens'); plt.xticks(rotation=0)
022 | plt.tight_layout(); plt.savefig(RESULTS / 'task3_categories.png', dpi=150); plt.show()
023 | save_table(category_words, 'task3_category_words.csv')
024 | save_table(category_totals, 'task3_category_totals.csv')
025 |
026 | # В PDF есть две формулировки редкости. Показываем обе явно.
027 | rare_per_genre = pd.DataFrame([{'genre': genre, 'word': word, 'count': count}
028 |     for genre, counts in genre_counts.items() for word, count in sorted(counts.items()) if 1 <= count
<= 3])
029 | display(Markdown('### Редкие слова отдельно для каждого жанра (1–3 вхождения)'))
030 | display(rare_per_genre.groupby('genre').size().to_frame('number_of_rare_words'))
031 | display(rare_per_genre.groupby('genre', sort=False).head(15))
```

```
save_table(rare_per_genre, 'task3_rare_per_genre_FULL.csv')

shared_vocabulary = set.intersection(*(set(c) for c in genre_counts.values()))
shared_rare_words = sorted(w for w in shared_vocabulary if all(1 <= genre_counts[g][w] <= 3 for g in
GENRES))
shared_rare = pd.DataFrame([{'word': w, **{g: genre_counts[g][w] for g in GENRES}} for w in
shared_rare_words],
                           columns=['word'] + GENRES)
display(Markdown('### Слова, встречающиеся во всех 4 жанрах и в каждом не более 3 раз'))
display(shared_rare)
if shared_rare.empty: print('Таких слов в выбранных 40 песнях нет — это допустимый результат.')
save_table(shared_rare, 'task3_shared_rare_FULL.csv')

top_categories = category_totals.loc[category_totals.groupby('genre').per_1000_tokens.idxmax()]
display(Markdown('**Вывод по выборке:**'))
for row in top_categories.itertuples():
    print(f'{row.genre}: наиболее частая из заданных категорий — {row.category}, '
          f'{row.per_1000_tokens:.1f} упоминаний на 1000 обработанных токенов.')
```

Блок 16. Задание 4: WiLI, Unicode и разбиение 80/20

Ячейка 30 в файле.

Строки	Объяснение
1–5	URL файлов WiLI, шесть кодов ISO3 и перевод ISO3→ISO2 для совместимости с fastText
7–8	minimal_clean нормализует Unicode и пробелы, сохраняя регистр, цифры и пунктуацию
10–13	aggressive_clean приводит к нижнему регистру, оставляет буквы любого алфавита и пробелы; остальные символы заменяет пробелами
15	Проверка: казахские и европейские специальные буквы не исчезают
16–19	Читаем оба исходных набора WiLI: x содержит тексты, y — соответствующие метки
20–23	zip_longest позволяет обнаружить несовпадающее число строк: недостающая сторона станет None; обычный zip молча остановился бы на коротком файле
24–27	Убираем перенос у метки, выбираем только шесть языков, сохраняем исходный split и номер строки
28–32	Создаём канонический ключ, отбрасываем меньше 3 слов, конфликтующие метки и дубликаты
33–37	Проверяем доступное количество, выбираем по 800 примеров на язык
38–42	Новое стратифицированное разбиение 80/20; проверяем непересечение и размер 3840/960
43–44	Показываем по 640 train и 160 test на язык, сохраняем выбранные примеры

`isalpha()` проверяет Unicode-буквы: в отличие от `[a-z]` не выбрасывает кириллицу. Не нужно удалять русские слова как «неанглийские» — в задаче определения языка их буквы являются полезными признаками. Лемматизация и удаление языковых стоп-слов здесь не нужны: `the`, `el`, `der` и окончания хорошо сигнализируют язык.

Мы **переопределяем исходное разбиение WiLI**, чтобы выполнить требование 80/20 на шести языках. Это учебный эксперимент, не официальный benchmark по неизменённому test WiLI. Перемешивание не смешивает наш итоговый train/test: новый раздельный test сформирован после очистки. Но принадлежность разных абзацев одной странице Wikipedia не учитывается — в этих файлах нет удобного ID страницы для группового разбиения. Это возможный источник слишком оптимистичной оценки переноса на совершенно новые источники.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | WILI_BASE = ('https://raw.githubusercontent.com/alianoroozi/Language-Identification/'
002 |              '9338c39f7dba99785bbedc7309013091f22d838c/wili-2018/')
003 | LANGUAGES = {'eng': 'English', 'rus': 'Russian', 'fra': 'French',
004 |              'deu': 'German', 'spa': 'Spanish', 'por': 'Portuguese'}
005 | ISO3_TO_ISO2 = {'eng': 'en', 'rus': 'ru', 'fra': 'fr', 'deu': 'de', 'spa': 'es', 'por': 'pt'}
006 |
007 | def minimal_clean(text):
008 |     return re.sub(r'\s+', ' ', unicodedata.normalize('NFKC', str(text))).strip()
009 |
010 | def aggressive_clean(text):
011 |     # Сохраняем ВСЕ Unicode-буквы: ə, қ, ғ, ñ, ü, é и т.д. не исчезают.
012 |     return re.sub(r'\s+', ' ', ''.join(ch if ch.isalpha() or ch.isspace() else ' '
013 |                                        for ch in minimal_clean(text).lower())).strip()
014 |
015 | assert aggressive_clean('Ə Қ Ғ Ң θ Ү é ñ 123 !') == 'ə қ ғ ң θ ү é ñ'
016 | wili_rows = []
017 | for split in ['train', 'test']:
018 |     x_path = fetch(WILI_BASE + f'x_{split}.txt', DATA / 'wili' / f'x_{split}.txt')
019 |     y_path = fetch(WILI_BASE + f'y_{split}.txt', DATA / 'wili' / f'y_{split}.txt')
020 |     from itertools import zip_longest
021 |     with open(x_path, encoding='utf-8-sig') as xf, open(y_path, encoding='utf-8-sig') as yf:
022 |         for line_no, (text, label) in enumerate(zip_longest(xf, yf)):
023 |             assert text is not None and label is not None, 'Число строк текстов и меток различается'
024 |             label = label.strip()
025 |             if label in LANGUAGES:
026 |                 wili_rows.append({'text': minimal_clean(text), 'label': LANGUAGES[label],
027 |                                   'iso3': label, 'source_split': split, 'source_line': line_no})
028 | wili = pd.DataFrame(wili_rows)
029 | wili['canonical'] = wili.text.map(aggressive_clean)
030 | wili = wili[wili.canonical.str.split().str.len() >= 3]
031 | label_counts = wili.groupby('canonical').label.nunique()
032 | wili = wili[~wili.canonical.isin(label_counts[label_counts > 1].index)].drop_duplicates('canonical')
033 | display(wili.label.value_counts().to_frame('available_unique_examples'))
034 | N_PER_LANGUAGE = 800
035 | assert wili.label.value_counts().min() >= N_PER_LANGUAGE
036 | language_data = pd.concat([group.sample(N_PER_LANGUAGE, random_state=SEED)
037 |                            for _, group in wili.groupby('label')], ignore_index=True)
038 | lang_train, lang_test = train_test_split(language_data, test_size=.2, random_state=SEED,
039 |                                          stratify=language_data.label)
040 | assert set(lang_train.canonical).isdisjoint(lang_test.canonical)
041 | assert set(lang_train.text).isdisjoint(lang_test.text)
042 | assert len(lang_train) == 3840 and len(lang_test) == 960
043 | display(pd.concat({'train': lang_train.label.value_counts(), 'test': lang_test.label.value_counts()},
```

```
axis=1))
044 | save_table(language_data.drop(columns='canonical'), 'task4_selected_data.csv')
```

Блок 17. Задание 4: восемь классических моделей

Ячейка 31 в файле.

Строки	Объяснение
1–2	Фиксируем порядок языков; создаём хранилища моделей, предсказаний и метрик
3–5	Три вложенных цикла: 2 очистки × 2 вида признаков × 2 модели = 8 экспериментов
6–8	Для char берём n-граммы 1–3 символа; для word — отдельные слова. min_df=2, максимум 60 000 признаков; lowercase=False отдаёт управление регистром нашей функции очистки
9–10	NB с alpha=.1 либо LR с C=3. Настройки заданы заранее
11–12	Собираем pipeline и обучаем на lang_train
13–18	Формируем понятное имя эксперимента, прогнозируем test, сохраняем модель и метрики
19–22	Печатаем ход выполнения; сортируем и сохраняем итоговую таблицу

Символьные признаки английского `ing`, португальского `ção` и русские буквы могут помогать даже для незнакомых слов. В режиме `char` минимальной очистки n-граммы могут включать пробелы и пунктуацию, а не только буквы. Словесная TF-IDF сильнее страдает от незнакомого словаря; стандартный шаблон токенизации слов здесь обычно игнорирует односимвольные слова.

Multinomial Naive Bayes оценивает, какие признаки типичны для каждого языка, и сочетает их с априорной вероятностью языка по формуле Байеса. «Наивный» — из-за предположения условной независимости признаков внутри класса. Оно приблизительно и часто неверно буквально, но метод всё равно полезен. `alpha` сглаживает нулевые оценки для невстречавшихся признаков. TF-IDF не является целочисленным счётчиком, но NB работает с такими неотрицательными весами как распространённая практическая базовая модель.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | LANG_LABELS = sorted(LANGUAGES.values())
002 | language_models, language_predictions, language_scores = {}, {}, []
003 | for cleaning, preprocessor in [('minimal', minimal_clean), ('aggressive', aggressive_clean)]:
004 |     for feature_type in ['char', 'word']:
005 |         for model_type in ['NB', 'LR']:
006 |             vectorizer = TfidfVectorizer(preprocessor=preprocessor, lowercase=False,
007 |                 analyzer=feature_type, ngram_range=(1, 3) if feature_type == 'char' else (1, 1),
008 |                 min_df=2, max_features=60000, sublinear_tf=True)
009 |             estimator = (MultinomialNB(alpha=.1) if model_type == 'NB' else
010 |                          LogisticRegression(C=3, max_iter=1000, random_state=SEED))
011 |             model = Pipeline([('tfidf', vectorizer), ('classifier', estimator)])
```

```
012 |             model.fit(lang_train.text, lang_train.label)
013 |             name = f'{cleaning}/{feature_type}/{model_type}'
014 |             predicted = model.predict(lang_test.text)
015 |             language_models[name] = model
016 |             language_predictions[name] = predicted
017 |             language_scores.append({'model': name, 'cleaning': cleaning, 'features': feature_type,
018 |                                     'classifier': model_type, **evaluate(lang_test.label, predicted)})
019 |             print(name, 'готово', flush=True)
020 | language_metrics = pd.DataFrame(language_scores)
021 | display(language_metrics.sort_values('macro_f1', ascending=False))
022 | save_table(language_metrics, 'task4_classical_metrics.csv')
```

Блок 18. Задание 4: предобученный fastText

Ячейка 33 в файле.

Строки	Объяснение
1–6	Подключаем библиотеку, задаём ожидаемый хеш и два источника одного файла: официальный сервер и зеркало
7–14	Пробуем загрузить и проверить файл; при ошибке источника пробуем следующий. break останавливает цикл при успехе; если ни один не сработал — явная ошибка
15–16	Загружаем готовые веса и строим перевод кодов в названия шести языков
18–21	Определяем функцию для списка текстов, чистим переносы, создаём накопители
22–25	Берём top-1, убираем префикс `__label__`, переводим код в имя; незнакомый код оставляем как other:xx, сохраняем вероятность
26	По флагу возвращаем либо метки, либо пару массивов «метки и уверенности»
28–34	Оцениваем baseline на том же test и добавляем девятую строку в таблицу

Мы **не обучаем fastText на наших 3840 примерах**. У модели уже есть знания о 176 языках, в том числе из внешних Wikipedia-данных. Возможное пересечение источников с WiLI не даёт считать это идеально независимым сравнением. Версия `.ftz` компактная и квантованная.

Классификация fastText — компактная обученная модель с признаками текста, не словарь VADER и не большая генеративная T5. Нельзя переносить все детали механизма subword из любой другой модели fastText на конкретный lid.176 без проверки его параметров.

Если истинный язык English, а baseline сказал Italian, это ошибка. Мы не подменяем её ближайшим из шести разрешённых ответов. Macro F1 усредняется по нашим шести целевым языкам, чтобы дополнительный предсказанный язык не менял число усредняемых классов.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | import fasttext
002 | FASTTEXT_SHA256 = '8f3472cfe8738a7b6099e8e999c3cbfae0dcd15696aac7d7738a8039db603e83'
```

```
FASTTEXT_URLS = [
    'https://dl.fbaipublicfiles.com/fasttext/supervised-models/lid.176.ftz',
    ('https://raw.githubusercontent.com/sherozshaikh/pdflangdetect/'
     '068fe2f45464a8124617dfe1b58fad8cf82443b4/src/pdflangdetect/_data/lid.176.ftz')]
ft_path = None
for url in FASTTEXT_URLS:
    try:
        ft_path = fetch(url, DATA / 'lid.176.ftz', expected_sha256=FASTTEXT_SHA256)
        break
    except (requests.RequestException, ValueError) as exc:
        print(f'Источник недоступен: {type(exc).__name__}. Пробуем проверенное зеркало.')
if ft_path is None: raise RuntimeError('Не удалось загрузить fastText. Нужен доступ к одному из источников.')
ft_model = fasttext.load_model(str(ft_path))
iso2_names = {ISO3_TO_ISO2[k]: v for k, v in LANGUAGES.items()}

def fasttext_predict(texts, with_confidence=False):
    labels, confidences = [], []
    for text in texts:
        safe = minimal_clean(text).replace('\n', ' ')
        label, probability = ft_model.predict(safe, k=1)
        iso2 = label[0].removeprefix('__label__')
        labels.append(iso2_names.get(iso2, f'other:{iso2}'))
        confidences.append(float(probability[0]))
    return (np.array(labels), np.array(confidences)) if with_confidence else np.array(labels)

ft_pred = fasttext_predict(lang_test.text)
language_predictions['fastText lid.176'] = ft_pred
all_language_metrics = pd.concat([language_metrics, pd.DataFrame([
    {'model': 'fastText lid.176', 'cleaning': 'minimal', 'features': 'pretrained',
     'classifier': 'fastText', **evaluate(lang_test.label, ft_pred)}])], ignore_index=True)
display(all_language_metrics.sort_values('macro_f1', ascending=False))
save_table(all_language_metrics, 'task4_all_metrics.csv')
```

Блок 19. Задание 4: сравнение очистки и анализ ошибок

Ячейка 34 в файле.

Строки	Объяснение
2–5	pivot раскладывает метрики на строки «признаки/классификатор» и колонки очистки; считаем aggressive−minimal
6–9	Для каждой пары объясняем знак разницы; очень маленькую разницу считаем равенством
12–14	Основной метод заранее зафиксирован как minimal/char/LR; берём сохранённую модель и её предсказания
15–18	Отчёт по шести языкам, матрица основной модели и отдельная матрица fastText, включающая его дополнительные ответы
20–24	Оставляем ошибки, группируем по направлению истинный→предсказанный, сортируем по количеству
25–32	Для трёх самых частых направлений показываем пример и возможное объяснение
33–36	Если направлений меньше трёх, не выдумываем их; сохраняем все ошибки и счётчики

Знак Δ: отрицательный означает, что aggressive хуже для данной пары признаков и классификатора. На этом test minimal лучше в 3 из 4 пар, но word/LR немного выигрывает от aggressive. Различия малы, статистический тест значимости не проводился. Поэтому «агрессивная очистка всегда плохая» — неверный вывод.

Основная модель сделала 9 ошибок из 960. Самые частые направления: Portuguese→English (4), Russian→English (2), French→English (1). В показанных примерах встречаются англоязычные библиографические записи внутри страниц других языков. Это может быть несоответствие метки происхождения и фактического языка фрагмента, а не неспособность распознать русские буквы. Не все ошибки относятся к Spanish↔Portuguese: близкая пара выбрана для проверки, но результат анализа нужно читать таким, каким его дал код.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | # Интерпретация эксперимента с предобработкой без заранее придуманного победителя.
002 | cleaning_comparison = language_metrics.pivot(index=['features', 'classifier'],
003 |                                              columns='cleaning', values='macro_f1')
004 | cleaning_comparison['aggressive_minus_minimal'] = cleaning_comparison.aggressive -
cleaning_comparison.minimal
005 | display(cleaning_comparison)
006 | for (feature_type, classifier), row in cleaning_comparison.iterrows():
007 |     delta = row.aggressive_minus_minimal
008 |     outcome = 'одинаковы' if abs(delta) < 1e-9 else ('aggressive лучше' if delta > 0 else 'minimal
лучше')
009 |     print(f'{feature_type}/{classifier}: {outcome}, Δ macro F1 = {delta:+.4f}')
010 |
011 | # Фиксированная основная модель выбрана ДО теста; таблица сравнения не меняет это решение.
012 | PRIMARY_LANGUAGE_MODEL = 'minimal/char/LR'
013 | primary_language_model = language_models[PRIMARY_LANGUAGE_MODEL]
014 | primary_lang_pred = language_predictions[PRIMARY_LANGUAGE_MODEL]
015 | print(classification_report(lang_test.label, primary_lang_pred, labels=LANG_LABELS, zero_division=0))
016 | show_confusion(lang_test.label, primary_lang_pred, LANG_LABELS, PRIMARY_LANGUAGE_MODEL)
017 | ft_cm_labels = LANG_LABELS + sorted(set(ft_pred) - set(LANG_LABELS))
018 | show_confusion(lang_test.label, ft_pred, ft_cm_labels, 'fastText lid.176 — all predicted labels')
019 |
020 | lang_error_frame = lang_test[['text', 'label']].assign(predicted=primary_lang_pred)
021 | lang_error_frame = lang_error_frame[lang_error_frame.label != lang_error_frame.predicted]
022 | frequent_pairs = (lang_error_frame.groupby(['label', 'predicted']).size().rename('errors')
023 |                   .reset_index().sort_values('errors', ascending=False))
024 | display(frequent_pairs.head(3))
025 | for row in frequent_pairs.head(3).itertuples():
026 |     example = lang_error_frame[(lang_error_frame.label == row.label) &
027 |                                (lang_error_frame.predicted == row.predicted)].iloc[0].text
028 |     print(f'\n{row.label} → {row.predicted}: {row.errors} ошибок. Пример: {example[:500]}')
029 |     if {row.label, row.predicted} == {'Spanish', 'Portuguese'}:
030 |         print('Возможная причина: близкая лексика, общий алфавит и похожие короткие сочетания букв.')
031 |     else:
032 |         print('Проверить цитаты на другом языке, имена, общую лексику и возможную ошибку разметки
Wikipedia.')
033 | if len(frequent_pairs) < 3:
034 |     print('На test встретилось меньше трёх типов ошибок; дополнительные ошибки не выдумываем.')
035 | save_table(lang_error_frame, 'task4_errors.csv')
036 | save_table(frequent_pairs, 'task4_error_pairs.csv')
```

Блок 20. Задание 4: длина текста и функция для пользователя

Ячейка 36 в файле.

Строки	Объяснение
1–6	Собираем канонические ключи обучающих текстов, предложений и трёхсловных префиксов, чтобы отфильтровать очевидные совпадения
7–14	Для каждого test-примера создаём вариант из первых 3 слов и вариант из первого предложения
15–19	Для длинного варианта берём первые 2 предложения. Если предложение одно, добавляем первое предложение другого test-текста того же языка; помечаем paired=True
20–25	Удаляем пустые/известные обучению варианты, записываем группу длины и число слов, удаляем повторные тексты внутри группы
26–27	Получаем ответы всех 8 моделей и fastText на одинаковом стресс-наборе
28–37	Для каждой модели и группы считаем число примеров, среднее число слов, accuracy и macro F1
38–44	Рисуем сравнение accuracy и сохраняем метрики и сами варианты
46–47	Функция identify_language отклоняет пустой текст
48–50	Получает вероятности LR, порядок классов и индекс самого большого значения
51–53	Добавляет ответ и вероятность fastText; возвращает удобный словарь
54–57	Показывает примеры на всех шести языках

`(i+1) % len(examples)` выбирает следующий текст циклически: после последнего возвращается к первому. `.argmax()` возвращает индекс максимума, не само максимальное число. Маска `mask` выбирает только нужную группу и одинаково применяется к меткам и предсказаниям.

В реальном запуске короткая группа состоит из **ровно трёх слов**, потому что исходники содержат минимум три. Название «1–3 words» — диапазон категории, а не доказательство, что мы отдельно проверили однословные и двухсловные входы. После фильтров получилось 820 / 945 / 960 примеров по трём группам; не надо сравнивать их как совершенно независимые равные выборки. Это связанные варианты одних test-текстов, а часть длинных вариантов искусственно объединена. Формальная оценка эффекта длины потребовала бы дополнительного продуманного дизайна эксперимента.

Вероятность 0.93 у LR не гарантирует 93% правильности на всех подобных сообщениях: вероятности не калибровались. Основная модель выбирает только из шести языков; казахский в её обучении отсутствует. Проверка сохранности казахских букв в очистке не добавляет казахский как седьмой класс. fastText может распознавать языки вне нашей шестёрки.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | train_text_keys = set(lang_train.text.map(aggressive_clean))
002 | train_sentence_keys = set()
003 | train_short_keys = set()
004 | for text in lang_train.text:
005 |     for sent in nltk.sent_tokenize(text): train_sentence_keys.add(aggressive_clean(sent))
006 |     train_short_keys.add(aggressive_clean(' '.join(text.split()[:3])))
007 | stress_rows = []
008 | for language, group in lang_test.groupby('label'):
009 |     examples = group.text.tolist()
010 |     for i, text in enumerate(examples):
011 |         sentences = nltk.sent_tokenize(text)
012 |         if not sentences: continue
013 |         variants = [('1–3 words', ' '.join(text.split()[:3]), False),
014 |                     ('1 sentence', sentences[0], False)]
015 |         if len(sentences) >= 2:
016 |             variants.append(('2+ sentences', ' '.join(sentences[:2]), False))
017 |         elif len(examples) > 1:
018 |             other = nltk.sent_tokenize(examples[(i + 1) % len(examples)])[0]
019 |             variants.append(('2+ sentences', sentences[0] + ' ' + other, True))
020 |         for bucket, variant, paired in variants:
021 |             key = aggressive_clean(variant)
022 |             if not key or key in train_text_keys or key in train_sentence_keys or key in
train_short_keys: continue
023 |             stress_rows.append({'label': language, 'bucket': bucket, 'text': variant,
024 |                                 'paired_test_sentences': paired, 'word_count': len(variant.split())})
025 | stress_data = pd.DataFrame(stress_rows).drop_duplicates(['bucket', 'text'])
026 | stress_predictions = {name: model.predict(stress_data.text) for name, model in language_models.items()}
027 | stress_predictions['fastText lid.176'] = fasttext_predict(stress_data.text)
028 | stress_scores = []
029 | for name, pred in stress_predictions.items():
030 |     for bucket in ['1–3 words', '1 sentence', '2+ sentences']:
031 |         mask = stress_data.bucket == bucket
032 |         if not mask.any(): continue
033 |         stress_scores.append({'model': name, 'bucket': bucket, 'n': int(mask.sum()),
034 |             'mean_words': float(stress_data.loc[mask, 'word_count'].mean()),
035 |             **evaluate(stress_data.loc[mask, 'label'], pred[mask.to_numpy()])})
036 | stress_metrics = pd.DataFrame(stress_scores)
037 | display(stress_metrics)
038 | plot_order = ['1–3 words', '1 sentence', '2+ sentences']
039 | stress_metrics.pivot(index='bucket', columns='model',
values='accuracy').reindex(plot_order).plot.bar(figsize=(14, 5))
040 | plt.ylabel('Accuracy'); plt.ylim(0, 1.05); plt.xticks(rotation=0)
041 | plt.legend(loc='lower right', fontsize=8); plt.tight_layout()
042 | plt.savefig(RESULTS / 'task4_length_accuracy.png', dpi=150); plt.show()
043 | save_table(stress_metrics, 'task4_length_metrics.csv')
044 | save_table(stress_data, 'task4_length_examples.csv')
045 |
046 | def identify_language(text):
047 |     if not minimal_clean(text): raise ValueError('Введите непустой текст')
048 |     probs = primary_language_model.predict_proba([text])[0]
049 |     classes = primary_language_model.named_steps['classifier'].classes_
050 |     j = int(probs.argmax())
051 |     ft_label, ft_conf = fasttext_predict([text], with_confidence=True)
052 |     return {'text': text, 'TFIDF_LR': classes[j], 'LR_probability_uncalibrated': float(probs[j]),
053 |             'fastText': ft_label[0], 'fastText_probability_uncalibrated': float(ft_conf[0])}
054 | display(pd.DataFrame([identify_language(s) for s in [
055 |     'Сегодня очень хорошая погода.', 'The weather is wonderful today.',
056 |     'Hoy hace muy buen tiempo.', 'Hoje o tempo está muito bom.',
057 |     'Il fait très beau aujourd’hui.', 'Heute ist das Wetter sehr schön.']]))
```

Блок 21. Журнал воспроизводимости и завершение

Ячейка 39 в файле.

Строки	Объяснение
2–4	Читаем фактические версии установленных библиотек через importlib.metadata, а не предполагаемые версии из команды pip
5–9	Создаём manifest: seed, Python, пакеты, URL/хеши, размеры разбиений, количество песен и хеши очищенных статей
10	Записываем JSON в UTF-8. ensure_ascii=False сохраняет кириллицу читаемой; indent=2 форматирует отступы
11–18	Показываем версии, папку результатов и числа обработанных объектов

Этот журнал помогает повторить и проверить работу. Он не сохраняет живые Python-объекты обученных моделей: после перезапуска ядра нужно выполнить обучение снова. `.ipynb` хранит код и выводы, но не всю оперативную память. Успешный print в последней строке не заменяет метрики: доказательство работы находится в результатах предыдущих блоков и их проверках.

Между блоками 20 и 21 расположены Markdown-вывод по языкам и ответы на 15 контрольных вопросов. Их не нужно «выполнять как код»: они объясняют результат и теорию. Более подробные варианты ответов и дополнительные вопросы для защиты приведены далее в этом документе.

Исходный код с номерами строк

Номера слева — для навигации; код для запуска берите из ноутбука.

```
001 | # Сохраняем происхождение данных, версии и сводный отчёт.
002 | packages = ['numpy', 'pandas', 'scikit-learn', 'nltk', 'transformers', 'torch',
003 |             'optimum', 'onnxruntime', 'fasttext-wheel', 'wordcloud', 'rouge-score']
004 | versions = {p: importlib.metadata.version(p) for p in packages}
005 | manifest = {'seed': SEED, 'python': sys.version, 'packages': versions, 'downloads': DOWNLOAD_LOG,
006 |             'task1_split': {'train': len(sent_train), 'validation': len(sent_val), 'test':
len(sent_test)},
007 |             'task3_songs': lyrics_sample.groupby('genre').size().to_dict(),
008 |             'task4_split': {'train': len(lang_train), 'test': len(lang_test)},
009 |             'article_sha256': {n: hashlib.sha256(t.encode()).hexdigest() for n, t in
documents.items()}}
010 | (RESULTS / 'run_manifest.json').write_text(json.dumps(manifest, ensure_ascii=False, indent=2),
encoding='utf-8')
011 | display(pd.Series(versions, name='version').to_frame())
012 | display(Markdown('### Выполнение завершено'))
013 | print('Результаты:', RESULTS.resolve())
014 | print('Сохранено файлов:', len(list(RESULTS.iterdir())))
015 | print('Task 1: test examples =', len(sent_test))
016 | print('Task 2: 3 × 3 extractive sentences; 6 combined; all 5 documents summarized')
017 | print('Task 3:', lyrics_sample.groupby('genre').size().to_dict())
018 | print('Task 4: 8 classical models + pretrained fastText; test examples =', len(lang_test))
```

6. Результаты проверенного запуска

Числа ниже взяты из сохранённых выводов ноутбука, а не из предполагаемых результатов. Результаты относятся к конкретной очистке, разбиению и настройкам. На другом корпусе или при другом seed они могут отличаться.

Задание 1: тональность

Истинный класс	Train	Validation	Test
Negative	5 409	1 803	1 804
Neutral	1 703	567	568
Positive	1 276	426	425
Всего	8 388	2 796	2 797

Метод	Accuracy	Macro F1
Всегда отвечать negative	64.50%	0.2614
VADER, пороги 0.0 / 0.4	55.85%	0.5326
TF-IDF + LR, C=2	76.87%	0.7057

Как объяснить: большинство отзывов отрицательные. Постоянный negative уже даёт 64.5% accuracy, но совершенно не находит neutral и positive. Поэтому VADER хуже этого baseline по accuracy, но заметно лучше по macro F1. LR обгоняет оба метода по обеим метрикам. У LR F1 negative ≈0.86, neutral ≈0.57, positive ≈0.69: нейтральный класс труднее различить. Нельзя сказать «точность всех классов 77%»: 76.87% — общая accuracy, а не recall каждого класса.

Задание 2: суммаризация

После очистки длины пяти статей: 426, 699, 636, 527 и 314 слов по `split()`. Для extractive части исходные три статьи содержат 1 761 слово, все пять — 2 602 слова. Получено по 3 извлечённых предложения из первых трёх статей и 6 предложений общего списка. T5 обработала 2 + 3 + 2 + 2 + 1 = **10 исходных токеновых блоков**.

Метод и его вход	Слов в сводке	Доля исходной длины	ROUGE-1 F1	ROUGE-2 F1	ROUGE-L F1
Извлечение, 3 статьи	137	7.78%	0.3153	0.0364	0.1892
T5, 5 статей	35	1.35%	0.1818	0.0000	0.0979

Настоящий вывод T5:

> melody pitch sequence is converted into a note sequence. similarity between input polyphonic music and those of music in the database is calculated. break originated from the 1969 Funk recording Amen Brother by The Winstons.

В переводе по смыслу: последовательность высот мелодии преобразуется в ноты, считается сходство с музыкой в базе, затем упоминается происхождение музыкального фрагмента из Amen Brother.

Что получилось плохо: пропущены MFCC/энтропия/энергия, индексирование, сэмплирование; слово break потеряло уточнение Amen; предложения начинаются со строчной буквы и соединены слабым переходом. Маленькая длина не равна хорошей полноте. Нулевой ROUGE-2 означает, что после применённой обработки не совпали биграммы с нашим эталоном; это не самостоятельное доказательство ложности каждого сгенерированного факта.

Выбранные extractive предложения тоже не идеальны: встречаются общие фразы о структуре работы, а похожие формулировки повторяются по смыслу. TF-IDF-центроид оценивает тематическую типичность, не умея отличать научный вклад от описания оглавления.

Честный вывод на защите: автоматический базовый метод реализован и проверен, но его качество ограничено. Авторская вычитанная аннотация в следующей Markdown-ячейке исправляет пропуски. Она не должна выдаваться за машинный результат и не участвовала в показанных ROUGE-метриках как кандидат. Самостоятельный качественный результат — результат редакторской проверки.

Задание 3: песни

Выбрано 40 разных текстов: country=10, blues=10, folk=10, hip-hop=10.

Жанр	Самое частое слово и число	Редких слов с частотой 1–3	Body parts на 1000 токенов
Country	know — 52	372	12.36
Blues	oh — 73	245	17.63
Folk	get — 31	370	29.84
Hip-hop	im — 65	629	6.88

В пересечении редких слов всех жанров найдено **15 слов**, в том числе alone, everything, fight, morning, smile. Morning имеет частоты 1 / 3 / 1 / 3 по country / blues / folk / hip-hop. Это пример редкости в каждом жанре, хотя суммарно слово встречается 8 раз.

Во всех четырёх жанрах наиболее частой **среди наших трёх категорий** оказалась body_parts. Нельзя превращать это в утверждение «все песни в основном о теле»: большая часть слов вообще не входит в заданные категории. И folk здесь имеет большую относительную частоту, но выборка из 10 песен не доказывает свойство всего жанра.

Задание 4: языки

Всего 4 800 примеров: 3 840 train, 960 test. На язык — 640 и 160 соответственно.

Очистка / признаки / модель	Accuracy	Macro F1
Minimal / char / NB	98.02%	0.9805
Minimal / char / LR — основной метод	99.06%	0.9907

Очистка / признаки / модель	Accuracy	Macro F1
Minimal / word / NB	99.06%	0.9907
Minimal / word / LR	98.85%	0.9886
Aggressive / char / NB	97.81%	0.9785
Aggressive / char / LR	98.85%	0.9886
Aggressive / word / NB	98.75%	0.9876
Aggressive / word / LR	98.96%	0.9897
FastText lid.176	98.12%	0.9820

Minimal/word/NB имеет чуть больший macro F1, чем minimal/char/LR, хотя после округления видно одно число. Основной метод зафиксирован заранее, а не выбран задним числом. Символьные признаки не выиграли безусловно у всех моделей на всех длинах — не нужно так утверждать. Разница в 1–2 ошибки на 960 объектах не доказывает универсального превосходства.

Основная LR правильно определила 951 из 960 текстов. Типичный ошибочный Russian→English пример — библиографическая запись `Thomson, Harry C.; Mayo, Lida. The Ordnance Department...` с небольшой русской пометой «англ.». Метка могла отражать русский раздел Wikipedia, хотя сам фрагмент в основном английский. Это объясняет, почему чтение ошибок полезнее одного итогового процента.

Метод	Первые 3 слова, n=820	Первое предложение, n=945	Два предложения, n=960
Minimal / char / LR	84.51%	98.52%	99.06%
Minimal / char / NB	90.24%	97.88%	98.23%
Minimal / word / LR	51.46%	96.19%	98.33%
FastText lid.176	88.17%	97.46%	98.12%

На трёх словах информации мало, многие слова общие для языков; длинные входы проще. Но n различается из-за фильтров и дубликатов, а варианты связаны: это наблюдение стресс-теста, не строгий эксперимент, изолирующий только длину. На коротких строках NB оказался устойчивее основной LR — ещё одна причина не судить о пригодности только по полным абзацам.

7. Вопросы преподавателя и ответы

Ниже ответы сформулированы так, чтобы их можно было произнести вслух. Понимайте смысл, а не пытайтесь заучить названия параметров без связи с экспериментом.

7.1. Пятнадцать контрольных вопросов из PDF

1. Чем extractive отличается от abstractive?

Extractive копирует выбранные предложения исходника. Abstractive создаёт новые формулировки, может перефразировать и соединять факты. В нашей работе первое делает TF-IDF-ранжирование, второе — T5. Генерация потенциально связнее, но может придумать ошибочные факты.

2. Почему TF-IDF/GraphRank выбирают предложения, а не отдельные слова?

Потому что в summarization мы назначили единицей отбора предложение: у него есть вектор или узел графа. Слова помогают оценить предложение. Набор отдельных важных слов ещё не является грамматически связной сводкой. Сам TF-IDF можно применить и к другим единицам. В нашем коде графового TextRank нет; использован центроид. TextRank ранжировал бы узлы графа сходства предложений итеративно, как PageRank.

3. Какие трудности у абстрактивного суммирования?

Нужно сохранить главное при ограниченной длине, связать разные документы, избежать повторов и не исказить факты. Длинный контекст приходится резать. В нашей иерархии каждый этап сжатия способен потерять детали; T5 действительно пропустила несколько важных методов.

4. Что такое ROUGE и как его применять?

Сравниваем текст кандидата с эталоном: отдельные слова, пары слов и общие подпоследовательности. Считаем precision, recall, F1 соответствующих совпадений. Это полезно для измерения лексического покрытия, но корректный парафраз может получить низкий балл, а фактическая ошибка с похожими словами — относительно высокий.

5. Почему важна грамматическая правильность генерации?

Модель пишет новые предложения. Ошибка в отрицании, связи частей или местоимении может не просто испортить стиль, а изменить смысл. Поэтому сохранение ключевых слов недостаточно. Экстрактивный метод берёт исходную грамматику, но тоже может потерять контекст.

6. В чём отличие словарного метода тональности от ML?

У VADER эмоциональные оценки слов и правила заданы заранее; у LR связи между признаками и классами изучены на наших размеченных отзывах. Словарный метод проще объяснить и запустить без разметки для обучения, но он может хуже учитывать домен. ML требует разметки и проверки на новых данных. У нас для VADER всё же использовалась разметка validation при подборе порогов.

7. Зачем лемматизация?

Чтобы eyes и eye, flights и flight учитывались как одно базовое слово, уменьшая разреженность и раздробление частот. Сначала желательно знать часть речи: одинаковая запись может иметь разные грамматические функции. Лемматизация не исправляет автоматически любой сленг и опечатки.

8. Что показывает confusion matrix?

Сколько объектов каждого истинного класса модель направила в каждый предсказанный. По диагонали — правильные решения, вне её — типы ошибок. В нашей матрице языка видно, что часть Portuguese и Russian попала в English. Порядок осей всегда нужно проверить.

9. Почему accuracy может быть обманчивой?

Из-за дисбаланса. Постоянный negative в наших отзывах даёт 64.5%, но не распознаёт два других класса. Macro F1 показывает этот провал: 0.2614. Нужно читать также per-class precision/recall, а не только одну метрику.

10. Что мешает определению тональности?

Сарказм, отрицания, смешанные эмоции, доменные значения, краткость, сленг и шумная разметка. В примере `Great, another three-hour delay` словарь видит положительное great, но смысл жалобы отрицательный. Нельзя гарантировать, что ML поймёт любой такой случай.

11. Чем нормализация отличается от стандартизации?

В числовых признаках стандартизация обычно означает `(x−mean)/std`. Нормализация может означать масштабирование в [0,1] либо единичную норму вектора. В NLP нормализация текста — другое употребление слова: регистр, Unicode, словоформы. В ноутбуке есть текстовая нормализация и стандартное L2-нормирование TF-IDF; StandardScaler для z-score мы не применяли.

12. Чем TF-IDF лучше простых частот?

Обычная частота может выделять повсеместные слова, плохо различающие документы. IDF уменьшает их относительный вклад, оставляя больше веса более характерным словам. Однако это не универсальное превосходство: для задачи простого подсчёта упоминаний в песнях нам как раз нужны частоты, а не TF-IDF.

13. Почему генеративная модель может ошибаться грамматически?

Она выбирает вероятные продолжения на основе обучения, а не проверяет формальную теорему о грамматике. Мешают шумные источники, длинные зависимости, небольшой размер модели, ограничения декодирования и смена домена. Квантование тоже может влиять на ответы, но в нашем эксперименте его отдельный эффект не измерялся.

14. Чем ROC отличается от Precision–Recall?

ROC строит TPR=recall против FPR при изменении порога. PR строит precision против recall. При очень редком положительном классе PR часто лучше показывает цену ложных срабатываний: FPR делится на большое число отрицательных объектов и может выглядеть маленьким. Наш ноутбук эти кривые не строит: в нём требуемые accuracy, macro F1 и confusion matrix. Для нескольких классов нужен, например, one-vs-rest и явный способ усреднения.

15. Что такое Bag-of-Words?

Документ превращается в вектор количеств слов из фиксированного словаря. Порядок слов обычно теряется: «dog bites man» и «man bites dog» имеют одинаковые количества. Биграммы частично сохраняют локальный порядок; TF-IDF меняет веса тех же признаков. В нашем анализе тональности используются униграммы и биграммы с TF-IDF.

7.2. Дополнительные вопросы по нашей реализации

Почему вы взяли авиакомпании, а не IMDb?

Задание допускает похожий корпус отзывов. У авиакомпаний сразу есть три размеченных класса, включая neutral. Обычный IMDb чаще бинарный, и без новой разметки нельзя честно научить и оценить нейтральный класс. Наши выводы относятся к клиентским сообщениям об авиакомпаниях.

Что такое обучаемый параметр и гиперпараметр?

Обучаемые параметры — коэффициенты LR, которые оптимизатор находит по train. Гиперпараметры задаются до fit: C, число n-грамм, min_df, alpha, число beams. Мы выбрали C по validation; веса не задавали вручную. VADER-пороги — настройки решения, а не изученные веса словаря.

Можно ли подбирать C по test, если так получится выше результат?

Тогда test участвует в выборе модели и перестаёт быть независимым итоговым экзаменом. Лучше validation или cross-validation на обучающей части. Cross-validation в этом ноутбуке не выполнялась. Мы не утверждаем, что один split исчерпывающе оценивает качество.

Зачем Pipeline, если можно вызвать TF-IDF отдельно?

Pipeline связывает векторизацию и классификатор: fit обучает их по порядку, predict преобразует новый текст тем же словарём. Это уменьшает риск перепутать матрицы или обучить векторизатор на test. Но Pipeline не защищает от всех ошибок, если передать ему уже содержащие утечку данные.

Почему не убрали not вместе со стоп-словами?

Потому что «not good» и «good» часто имеют разную тональность. Стандартный стоп-список содержит некоторые отрицания; мы исключаем их из удаляемого множества. При подсчёте тематической лексики песен отрицания не сохраняются отдельно: там другая цель.

Если good положительное, почему отзыв со словом good может быть negative?

`Not good`, ирония, цитирование и жалоба на другую часть услуги меняют смысл. VADER применяет правила отрицания, LR может выучить биграммы, но оба подхода ограничены.

Почему C меньше — регуляризация сильнее?

Так определён параметр в scikit-learn: C обратно связан с силой штрафа за большие веса. Большой C разрешает сильнее подстроиться под данные, маленький сильнее ограничивает модель. Это не «процент уверенности» и не «число классов».

Почему лучший macro F1 может не совпасть с лучшей accuracy?

Accuracy считает все примеры поровну; macro F1 — классы поровну. Модель может улучшить качество редкого класса ценой нескольких ошибок большинства. Выбор метрики зависит от цели; в нашем дисбалансном sentiment macro F1 был разумным критерием настройки.

Как работает логистическая регрессия простыми словами?

Каждый признак умножается на выученный вес; суммы и смещения дают оценки классов. Для многоклассовой версии они преобразуются softmax в вероятности. Обучение меняет веса, чтобы правильные ответы получали больше вероятности, с учётом регуляризации и весов классов. Несмотря на линейное решение, тысячи TF-IDF-признаков делают его хорошей базовой моделью.

Почему Naive Bayes называется наивным?

Он упрощённо предполагает независимость признаков при известном классе. Для языка символьные сочетания на самом деле зависимы, но модель всё равно часто хорошо работает. Сглаживание alpha не даёт невстречавшемуся признаку обнулить всю оценку класса.

Означают ли большие коэффициенты LR, что эти слова вызывают эмоцию?

Нет. Они показывают статистическую связь признаков с классами в этом обучении. На них влияют соседние признаки, очистка и распределение данных. Это не причинное исследование.

Почему сводку не сделать из первых 512 токенов всех статей?

Тогда последующие документы могли бы вообще не попасть в модель. Иерархическая обработка даёт каждой статье пройти генератор, но не гарантирует, что все её темы сохранятся в финале. Это подтвердилось в нашем результате: обработать документ и полно отразить его — разные вещи.

Чем beam search отличается от случайной генерации?

Beam search сохраняет несколько наиболее перспективных продолжений и сравнивает их. Sampling случайно выбирает продолжение из распределения с настройками случайности. У нас do_sample=False, beams=4. Это повышает повторяемость, но не доказывает фактическую правильность и не означает, что выбрана глобально лучшая возможная сводка.

Почему выбрана T5-small, а не большая модель?

Она доступна для CPU и умеренной памяти. Использован компактный ONNX-вариант. Это компромисс ресурсов и качества, а не утверждение, что маленькая T5 лучшая для научных многодокументных сводок. Для улучшения можно проверить более сильную summarization-модель, добавив оценку на независимых эталонах.

Вы сами обучили T5 и fastText?

Нет. Мы скачали готовые веса и выполняем inference. Обучаемыми в нашем эксперименте являются LR и NB. Разделение «обучил сам / использовал предобученное» нужно говорить прямо.

Если T5 ошиблась, зачем нужна ручная аннотация?

Она показывает обнаруженные пропуски и пример качественной редакторской версии по источникам. Автоматический результат сохранён отдельно и оценён именно как автоматический. Ручная редактура не маскирует плохую метрику и не доказывает, что генератор достиг нужного качества.

Что можно улучшить в extractive методе?

MMR уменьшит смысловые повторы; тематические квоты помогут покрыть разные статьи; фильтрация фраз об оглавлении уберёт малоинформативные предложения; TextRank предложит другую схему ранжирования. Всё это предложения улучшения, не функции текущего ноутбука. Оценивать улучшения нужно по фактическому результату, а не обещанию.

Почему просто не сравнить ROUGE двух сводок и выбрать победителя?

У них разные источники и эталоны: extractive на 3 статьях, abstractive на 5. Для прямого сравнения нужны одинаковые входы, одинаковые эталоны и процедура оценки. Дополнительно нужны полнота, связность и проверка фактов, потому что ROUGE их полностью не измеряет.

Что такое MFCC? Мы его вычисляли?

Это коэффициенты, описывающие спектральные свойства аудиосигнала с учётом мел-шкалы. В наших статьях они упоминаются как признаки аудиоотпечатков. В ноутбуке мы обрабатываем текст статей, а не аудиофайлы; MFCC сами не вычисляем. MIDI описывает музыкальные события, а PCM — цифровые отсчёты звуковой волны: это контекст статьи, не наш отдельный аудиопроект.

Почему одного словаря категорий недостаточно для понимания песен?

Словарь считает написание/лемму, не смысл. Heart может быть метафорой, spring — не временем года. Контекстное определение смысла потребовало бы более сложной модели или ручной проверки.

Почему редкие слова не обязательно «интересные»?

Редкость здесь — только частота ≤3 в выбранном жанре. Имена, опечатки и варианты сленга тоже редкие. Нельзя автоматически объявлять их литературно значимыми.

Можно ли по этим 40 песням описать весь hip-hop или folk?

Нет. По 10 текстов — маленькая учебная выборка. Влияют исполнители, эпоха, длина и припевы; в исходной CSV нет всех полей для контроля этих факторов. Наши выводы — о выбранной совокупности.

Почему в задаче языка не лемматизировали тексты?

Характерные окончания, служебные слова и буквы помогают определить язык. Лемматизация может удалить часть сигналов и требует заранее выбрать языковой инструмент, хотя язык как раз неизвестен. Поэтому сравниваем две более общие очистки.

Зачем сохранять ə, қ, ñ и é?

Это характерные буквы и полезные сигналы. Фильтр только a–z разрушил бы кириллицу и диакритику. NFKC не является переводом алфавита в ASCII и не должна удалять эти буквы.

Почему fastText ниже нашей LR на этом test?

LR решает узкую задачу из шести сбалансированных языков, fastText — задачу из 176, и его не ограничивали шестью ответами. Различаются обучение и источники. Один test не доказывает, что узкая LR лучше fastText на любых языках и коротких шумных сообщениях.

В примере задания есть казахский. Почему ваша LR не говорит Kazakh?

В условии пример входа иллюстрирует задачу; для эксперимента выбраны шесть языков с похожей парой Spanish/Portuguese. Казахского в наших train-метках нет, поэтому LR не может выдать его как обученный класс. Можно добавить корпус казахского и переобучить, либо использовать более широкий baseline. Нельзя назвать русскую метку на казахском вводе успехом.

Что произойдёт с совершенно неизвестными словами?

Словесный TF-IDF не добавит их в уже обученный словарь. Если все признаки окажутся нулевыми, классификатор всё равно выдаст класс по своим базовым оценкам. Символьные n-граммы часто сохраняют часть информации незнакомого слова. Поэтому короткие сообщения требуют отдельной проверки и, возможно, отказа от ответа.

Что значит уверенность 0.9?

Это выходная оценка модели. Чтобы трактовать её как надёжную частоту правильных ответов, нужна калибровка и независимая проверка. В коде названия полей прямо содержат uncalibrated. У неизвестного языка модель тоже может выдать высокую, но ошибочную оценку.

Можно ли доверять графику «короткий/длинный» как строгому доказательству?

Он полезен как стресс-тест, но группы после фильтров различаются, примеры связаны, а часть длинных входов склеена из двух test-примеров одного языка. Кроме того, sent_tokenize вызывается с настройками по умолчанию, а не с отдельной настройкой разбиения для каждого языка; сокращения и библиография могут делиться неидеально. Это ограничения протокола.

Какие данные вы не должны показывать в обучении?

Test-метки при настройке моделей и статистику теста при fit векторизатора. Для чистой итоговой оценки также нельзя бесконечно улучшать код, глядя на тот же test. В реальной разработке после такого анализа нужен новый независимый тест или внешняя проверка.

Что вы сделали бы следующим шагом?

Сначала улучшил бы разметку и диагностику: проверка цитат WiLI, больше песен, качественные независимые эталоны сводок. Затем сравнил бы расширенную обработку сокращений, контекстный sentiment, разнообразие extractive и более сильную генерацию. Новые настройки выбирал бы на validation, а окончательное сравнение делал бы на отдельном test.

8. План выступления и тренировка

Короткое выступление на 3–5 минут

Пример формулировки, которую нужно адаптировать под собственное понимание:

> В работе четыре задачи NLP. Для тональности я использовал размеченные отзывы клиентов авиакомпаний, потому что там сразу есть negative, neutral и positive. После удаления повторов разделил данные на train, validation и test. Сравнил словарь VADER с TF-IDF и логистической регрессией. Пороги и C выбирались по validation, итоговые метрики — по test. LR получила около 76.9% accuracy и 0.706 macro F1. Macro F1 важен из-за дисбаланса классов.
>
> Для суммаризации я ранжировал предложения первых трёх статей по сходству с общим TF-IDF-центроидом и выбрал по три, затем общий top-6. Для пяти статей применил готовую T5-small с иерархическим сжатием, чтобы обработать все документы, а не только начало. Результат T5 оказался слишком коротким и неполным, поэтому в отчёте показаны реальные пропуски и отдельная авторская редактура. ROUGE использован как ограниченная лексическая оценка.
>
> Для песен взято по десять уникальных текстов четырёх жанров. После лемматизации посчитаны наиболее частые слова, категории и два варианта редкости. Частоты категорий нормированы на тысячу токенов; выводы относятся к этой маленькой выборке.
>
> Для языка я взял по 800 примеров WiLI для шести языков, включая испанский и португальский. Сравнил две очистки, словесный и символьный TF-IDF, NB и LR, а также готовый fastText. Основная символьная LR дала 99.06% accuracy на 960 тестовых примерах, но на трёх словах качество заметно снизилось. Часть ошибок связана с английскими цитатами внутри страниц других языков. Основной вывод: нужно анализировать данные и ошибки, а не только проценты.

Что показывать преподавателю в ноутбуке

1. Блок 4: распределение трёх классов и размеры split.
2. Блоки 5–6: выбор порогов/C, таблицу метрик и confusion matrix.
3. Блок 7: пример с great и задержкой рейса; не заявлять, что сарказм полностью решён.
4. Блоки 10 и 12: извлечённые предложения и настоящий вывод T5.
5. Блок 13 и пояснения после него: ROUGE, пропуски, ручную редактуру как отдельный результат.
6. Блоки 14–15: проверку 4×10, top-10, относительные категории, 15 общих редких слов.
7. Блоки 17–20: 8+1 моделей, ошибки по языковым парам и падение качества на коротких текстах.

Полезные небольшие эксперименты для понимания

Эти примеры не изменяют обученные веса; запускать после Run all:

```
english_tokens('The flights were not good!')
predict_sentiment('The service was good.')
predict_sentiment('The service was not good.')
identify_language('Hoje o tempo está muito bom.')
aggressive_clean('ə Ķ F Ң θ Y é ñ 123 !')
```

Посмотрите, где различаются словарь и ML. Совпадение двух ответов ещё не гарантирует истину; для оценки нужен независимый правильный ответ. Короткие тестовые фразы вручную нужны для понимания поведения, а не для замены всей тестовой выборки.

Самопроверка без подсказок

Попробуйте объяснить своими словами:

- Что содержится в text, processed, label и predicted?
- Где в коде TF-IDF учится, а где только применяется?
- Почему lower=0 и upper=0.4 дают neutral при score=0.4?
- Почему удалить дубликаты после split хуже, чем до него?
- Почему 64.5% accuracy у majority не делает его хорошим анализатором трёх классов?
- Чем одинаковое количество слов отличается от одинаковой относительной частоты?
- Откуда получились 3840 и 960?
- Почему число 512 относится к токенам, а не к словам?
- Почему ручная сводка не доказывает качество generate?
- Чем источник разметки Russian отличается от фактического языка английской цитаты?

Если эти пункты понятны, вы понимаете основу проекта. Не нужно запоминать каждую шестнадцатеричную сумму или точную последовательность импортов.

9. Запуск, файлы результатов и частые ошибки

Как запустить

1. Откройте notebook в Google Colab либо Jupyter с Python 3.10–3.12.
2. При первом запуске обеспечьте интернет для pip, GitHub-данных, ресурсов NLTK и моделей.
3. Выберите Run all. Для этой версии GPU и API-ключи не нужны.
4. Если NumPy уже был импортирован в несовместимой версии, после установки перезапустите ядро и снова Run all. Не продолжайте с частично несовместимыми библиотеками.
5. Убедитесь, что нет красных traceback и выполнена последняя кодовая ячейка.

На GitHub можно читать сохранённые выводы, но для исполнения нужен Jupyter/Colab. Если GitHub не отрисовывает большую .ipynb, это проблема предпросмотра, а не автоматический признак ошибки Python-кода. Данные и модели загружаются в текущую рабочую папку окружения.

Какие файлы появляются

Файл или группа	Что хранит
`nlp_assignment_data/`	Скачанные CSV, WiLI, ресурсы NLTK и модели; повторно не скачиваются без необходимости
`task1_metrics.csv`	Итоговые метрики трёх методов тональности
`task1_errors.csv`, `task1_top_features.csv`	Ошибки с гипотезами и признаки с весами
`task1_wordclouds.png`, `task1_feature_weights.png`	Облака слов и коэффициенты

Файл или группа	Что хранит
`task2_extractive.json`, `task2_combined_six.csv`	Сводки по документам и общий список
`task2_abstractive.txt`, `task2_chunk_summaries.csv`	Настоящая генерация T5 и промежуточные результаты
`task2_quality_metrics.csv`, `task2_document_coverage.csv`	Метрики сводок и лексический прокси покрытия
`task2_factual_support_review.csv`	Пары «сгенерированное → похожее исходное предложение» для ручной проверки
`task3_selected_songs.csv`	Номера строк выбранных песен и хеши
`task3_top10.csv`, `task3_category_*.csv`	Частоты слов и категорий
`task3_rare_per_genre_FULL.csv`, `task3_shared_rare_FULL.csv`	Полные списки двух видов редких слов
`task4_selected_data.csv`, `task4_all_metrics.csv`	Выбранные тексты и результаты всех 9 методов
`task4_errors.csv`, `task4_error_pairs.csv`	Ошибки основной языковой модели
`task4_length_metrics.csv`, `task4_length_examples.csv`	Метрики стресс-теста и соответствующие входы
`run_manifest.json`	Версии, источники, хеши и размеры выборок

Все `task*` и manifest сохраняются внутри `nlp_assignment_results/`. Отдельные CSV не обязаны лежать в GitHub рядом с notebook: код создаёт их при запуске; сохранённые выводы в самом .ipynb уже доступны для чтения.

Если появилась ошибка

Сообщение/ситуация	Что это значит и что делать
`NameError: ... is not defined`	Нужная верхняя ячейка не выполнена либо ядро перезапущено; запустите сверху
`ModuleNotFoundError`	Пакет отсутствует в текущем Python; проверьте установочную ячейку и выбранное ядро
NLTK `LookupError`	Не найден ресурс: выполните ячейку загрузки NLTK и проверьте сеть
HTTP 403/404, timeout	Доступ запрещён, URL недоступен либо сетевая ошибка; прочитайте точный адрес и статус, не подменяйте данные фиктивным набором
Не совпала SHA-256	Файл отличается от ожидаемого; не отключайте проверку, удалите повреждённый кеш и получите корректную копию

Сообщение/ ситуация	Что это значит и что делать
Предупреждение длинного tokenizer.encode	В этом коде длинный текст сначала считается/нарезается; перед моделью есть assert. Если упал именно assert, нужно исправить chunking, а не молча обрезать всё
`IProgress not found`	Нет виджета красивого прогресса; само вычисление может успешно продолжаться
`ConvergenceWarning`	Оптимизатор не достиг условия сходимости за max_iter; нужны диагностика и, возможно, увеличение лимита, а не скрытие предупреждения
Красивая таблица, но странные результаты	Проверьте источник, метки, предобработку, дисбаланс и примеры ошибок; отсутствие исключения не доказывает смысловую правильность

Что не стоит говорить на защите: «99% означает, что модель понимает любой язык», «T5 сама написала ручную аннотацию», «ROUGE проверяет истинность», «высокая частота доказывает смысл слова», «модель не ошибается». Корректная защита показывает, что вы понимаете назначение каждого шага, реальные результаты и пределы сделанных выводов.